

The Ultimate Data Handling Cheat Sheet

Parsa

2026-09-21

Contents

1	How to Use This Cheat Sheet	5
2	Part 0 — The 30-Second Exam Strategy	6
3	Part 1 — Foundations	6
3.1	1.1 What is a data frame?	6
3.2	1.2 Tidy data principles	6
3.3	1.3 The names of things	7
3.4	1.4 Base data.frame vs tibble	7
4	Part 2 — Creating Data	8
4.1	2.1 From vectors	8
4.2	2.2 From <code>tibble()</code>	8
4.3	2.3 From a matrix	8
4.4	2.4 From a matrix of counts (exam Q22-3 style)	9
4.5	2.5 From lists (mixed types)	9
4.6	2.6 With <code>tribble()</code> (row-wise, easy to read)	10
4.7	2.7 Sequences and repeats	10
5	Part 3 — Import	11
5.1	3.1 Excel (<code>.xls</code> , <code>.xlsx</code>)	11
5.2	3.2 SPSS (<code>.sav</code>)	11
5.3	3.3 SAS / Stata	11
5.4	3.4 Text files	11
5.5	3.5 From clipboard	11
5.6	3.6 From a URL	12
5.7	3.7 After reading — always run these five	12
6	Part 4 — Inspection in Detail	12
6.1	4.1 Structure	12
6.2	4.2 Summary statistics	13
6.3	4.3 Unique values	14
6.4	4.4 Missing values overview	14
6.5	4.5 Quick views of the top and bottom	15
7	Part 5 — Data Types	15
7.1	5.1 The seven types you'll meet	15
7.2	5.2 Numeric	16
7.3	5.3 Character	16
7.4	5.4 Logical	17
7.5	5.5 Dates	17

7.6	5.6 Date-times	17
7.7	5.7 Checking types	18
8	Part 6 — Type Conversion	18
8.1	6.1 Explicit conversion	18
8.2	6.2 The dangerous conversion	18
8.3	6.3 Character → factor → numeric trap	19
8.4	6.4 Character → Date	19
8.5	6.5 Logical → numeric	19
8.6	6.6 Conversion summary table	19
9	Part 7 — Factors and Reference Levels	20
9.1	7.1 Why factors matter	20
9.2	7.2 Explicit levels (setting the reference)	20
9.3	7.3 Change the reference later	20
9.4	7.4 Check dummy coding	20
9.5	7.5 Numeric-coded groups (Q1 style)	21
9.6	7.6 Ordered factors (ordinal data)	21
9.7	7.7 Manual dummies	21
9.8	7.8 Recode factor levels	22
9.9	7.9 Collapse rare levels	22
10	Part 8 — Missing Values	22
10.1	8.1 Detecting	22
10.2	8.2 Removing rows with NA	23
10.3	8.3 Replacing NA	24
10.4	8.4 Filling (LOCF — last observation carried forward)	24
10.5	8.5 Interpolation	25
10.6	8.6 <code>na.rm = TRUE</code> in functions	25
10.7	8.7 NA vs NaN vs NULL vs Inf	26
10.8	8.8 Report template	26
11	Part 9 — Column Operations	27
11.1	9.1 Select columns	27
11.2	9.2 Rename columns	28
11.3	9.3 Rename all columns	28
11.4	9.4 Reorder columns	29
11.5	9.5 Extract a column as a vector	29
11.6	9.6 Base R alternatives	30
12	Part 10 — Row Operations	31
12.1	10.1 Filter rows	31
12.2	10.2 Slice rows by position	32
12.3	10.3 Distinct (unique) rows	33
12.4	10.4 Sort rows	34
12.5	10.5 Base R alternatives	35
12.6	10.6 Row counts before and after	36
13	Part 11 — Creating and Modifying Columns	36
13.1	11.1 Mutate — add or modify columns	36
13.2	11.2 <code>ifelse</code> vs <code>if_else</code>	37
13.3	11.3 <code>case_when</code> (multiple conditions)	37
13.4	11.4 <code>cut</code> (bin numeric into categories)	38
13.5	11.5 Rownames	38
13.6	11.6 Transmute (keep only new columns)	39

13.7	11.7 across — apply to many columns	39
14	Part 12 — Grouped Operations	39
14.1	12.1 group_by + summarise	39
14.2	12.2 Summarise multiple columns	40
14.3	12.3 Count per group	40
14.4	12.4 Grouped mutate (add group-level summary)	41
14.5	12.5 Grouped filter (keep rows satisfying group condition)	41
14.6	12.6 Ungroup	42
14.7	12.7 Base R alternatives	42
14.8	12.8 Report template	43
15	Part 13 — Window Functions	43
15.1	13.1 Lag and lead	43
15.2	13.2 Cumulative functions	43
15.3	13.3 Ranking	44
15.4	13.4 n() and n_distinct()	44
15.5	13.5 first, last, nth	45
16	Part 14 — Reshaping	45
16.1	14.1 Wide to long	45
16.2	14.2 Long to wide	46
16.3	14.3 Separate one column into two	46
16.4	14.4 Unite two columns into one	47
16.5	14.5 Complete (fill in missing combinations)	47
16.6	14.6 Long vs wide: which to use	47
16.7	14.7 Report template	48
17	Part 15 — Combining Data Frames	48
17.1	15.1 Stack rows (union)	48
17.2	15.2 Stack columns (side-by-side)	49
17.3	15.3 The six joins	49
17.4	15.4 Joining by different key names	50
17.5	15.5 Join summary	50
17.6	15.6 Report template	51
18	Part 16 — Frequency and Contingency Tables	51
18.1	16.1 One-way frequency	51
18.2	16.2 Two-way contingency table	51
18.3	16.3 Proportions	51
18.4	16.4 Chi-square test	52
18.5	16.5 CrossTable (gmodels package)	53
18.6	16.6 Report template	53
19	Part 17 — Descriptive Statistics by Group	53
19.1	17.1 Standard summary table	53
19.2	17.2 Custom summary with SE and CI	54
19.3	17.3 Summary by multiple groupings	54
19.4	17.4 Formatting for a report	55
19.5	17.5 Base R alternatives	55
20	Part 18 — Sampling and Splitting	55
20.1	18.1 Simple random sample	55
20.2	18.2 Train/test split	56
20.3	18.3 Stratified sample (preserve group proportions)	56

20.4	18.4	K-fold cross-validation indices	56
20.5	18.5	Report template	57
21	Part 19	— Outliers	57
21.1	19.1	IQR rule	57
21.2	19.2	Standardized residuals	57
21.3	19.3	Remove outliers	58
21.4	19.4	Winsorize (cap at percentiles)	58
21.5	19.5	Report template	58
22	Part 20	— Strings	59
22.1	20.1	Basic operations	59
22.2	20.2	Detect, extract, replace	59
22.3	20.3	stringr (tidyverse)	60
22.4	20.4	Common cleaning tasks	60
22.5	20.5	Extract with regex	61
22.6	20.6	Report template	61
23	Part 21	— Dates and Times	61
23.1	21.1	Create dates	61
23.2	21.2	Components	61
23.3	21.3	Arithmetic	62
23.4	21.4	Compute age from birth date	62
23.5	21.5	lubridate (easier)	63
23.6	21.6	Common formats	63
24	Part 22	— Plots for Exploration	63
24.1	22.1	Histogram	63
24.2	22.2	Boxplot by group	64
24.3	22.3	Scatterplot	65
24.4	22.4	Bar chart	66
24.5	22.5	Density plot	67
24.6	22.6	Facets	68
24.7	22.7	QQ plot (normality check)	69
24.8	22.8	Base R alternatives	70
24.9	22.9	Report template	74
25	Part 23	— Export	74
25.1	23.1	CSV	74
25.2	23.2	Excel	74
25.3	23.3	RDS (R's native format)	74
25.4	23.4	RData (multiple objects)	75
25.5	23.5	Text file	75
25.6	23.6	Report tables	75
26	Part 24	— Full Workflow Template	75
27	Part 25	— Common Exam Scenarios	76
27.1	25.1	Excel with header rows to skip	76
27.2	25.2	Numeric-coded groups → factor with custom reference	76
27.3	25.3	Group-level summary table	76
27.4	25.4	Contingency table with percentages	77
27.5	25.5	Train / test split	77
27.6	25.6	Repeated measures wide → long	78
27.7	25.7	Treat a specific value as missing	78

27.8 25.8 Duplicate rows	78
27.9 25.9 Wide-to-long with multiple value columns	79
27.1025.10 Left-join with missing keys	79
27.1125.11 Rounded count table	80
28 Part 26 — Quick Reference Table	80
29 Part 27 — Theory Q&A (What to Say)	81
30 Part 28 — Common Mistakes	82
31 Part 29 — The Single Most Important Idea	82
31.1 What was added and why	83

1 How to Use This Cheat Sheet

Every data-handling exam question falls into one of these. Find your question, jump to the section.

Question type	Section	Typical phrasing
Create a data frame	3	“Enter the data as a data frame”
Import a file	4	“Read Q1.xls into R”
Inspect the data	5	“Describe the structure of the data”
Convert column types	6	“Treat group as a factor”
Set the reference level	7	“Use control as reference”
Handle missing values	8	“Two values are missing. What do you do?”
Select / rename / reorder columns	9	“Keep only id and score”
Filter rows	10	“Subset the men only”
Add / modify columns	11	“Create a BMI column”
Group summaries	12	“Report mean score by group”
Window / lag / cumulative	13	“Add a column of cumulative totals”
Reshape wide → long	14	“Convert to long format for repeated measures”
Combine data frames	15	“Merge the two tables by id”
Frequency / contingency	16	“Make a 2-way table with percentages”
Descriptive stats by group	17	“Produce a summary table”
Train / test split	18	“Split 80/20”
Outliers	19	“Identify and remove outliers”
String cleaning	20	“Fix inconsistent labels”
Dates	21	“Compute age from date of birth”
Plots	22	“Plot the distribution by group”
Export	23	“Save the cleaned data as .csv”

Golden rule of this document: every chunk is self-contained. Each one starts with `set.seed()` where random, builds its own data, and uses section-prefixed names (`p3_df`, `p7_fac`, ...).

2 Part 0 — The 30-Second Exam Strategy

For **any** data-handling question, run this pipeline:

1. **Read** the data (`read_excel`, `read_csv`, `read_sav`).
2. **Inspect** (`dim`, `str`, `head`, `colSums(is.na())`).
3. **Fix types** (`as.numeric`, `factor(levels = ...)`).
4. **Check missing values** (`sum(is.na())`, decide: drop or fill).
5. **Clean / transform** (`filter`, `mutate`, `pivot_longer`).
6. **Summarise** (`group_by`, `summarise`).
7. **Verify** (`nrow` before vs after, spot-check rows).
8. **Report** in words: what you did, why, and what the output means.

If you can do these eight steps, you can handle any data-handling question in the exam.

3 Part 1 — Foundations

3.1 1.1 What is a data frame?

A data frame is a **table**: - **Rows**: observations (people, days, units). - **Columns**: variables (age, group, score). - Each column is a vector of the **same length**. - Columns can be of **different types**.

```
# --- Self-contained: anatomy of a data frame ---
p1_df <- tibble(
  id      = 1:4,
  gender  = c("M", "F", "M", "F"),
  age     = c(25, 30, 22, 28),
  score   = c(78, 85, 90, 72)
)
p1_df
```

```
## # A tibble: 4 x 4
##       id gender  age score
##   <int> <chr> <dbl> <dbl>
## 1     1   M      25     78
## 2     2   F      30     85
## 3     3   M      22     90
## 4     4   F      28     72
```

The output shows: - # A tibble: 4 × 4 → 4 rows, 4 columns. - Each column name is followed by its type (<int>, <chr>, <dbl>).

3.2 1.2 Tidy data principles

A data frame is **tidy** if: 1. Each **variable** has its own column. 2. Each **observation** has its own row. 3. Each **value** has its own cell.

Wide (not tidy):

```
id y0 y1 y2
1  10 11 12
```

Long (tidy):

```
id time y
1    0  10
1    1  11
```

1 2 12

Most tidyverse functions expect tidy data.

3.3 1.3 The names of things

```
# --- Self-contained: naming pieces ---  
p1_df <- tibble(id = 1:3, score = c(10, 20, 30))
```

```
p1_df$score          # column (a vector)
```

```
## [1] 10 20 30
```

```
p1_df[[1]]           # first column (a vector)
```

```
## [1] 1 2 3
```

```
p1_df[["score"]]      # column by name (a vector)
```

```
## [1] 10 20 30
```

```
p1_df[, "score"]      # tibble with one column
```

```
## # A tibble: 3 x 1
```

```
##   score
```

```
##   <dbl>
```

```
## 1    10
```

```
## 2    20
```

```
## 3    30
```

```
p1_df[1, ]           # first row (a tibble)
```

```
## # A tibble: 1 x 2
```

```
##     id score
```

```
##   <int> <dbl>
```

```
## 1     1    10
```

```
p1_df[1, 2]           # element in row 1, column 2
```

```
## # A tibble: 1 x 1
```

```
##   score
```

```
##   <dbl>
```

```
## 1    10
```

Rule: \$ and [[extract the **contents**; [extracts a **subset** (still a tibble).

3.4 1.4 Base data.frame vs tibble

Feature	data.frame	tibble
Print	All rows	First 10 rows, with types
Strings	Auto-factor (R < 4.0)	Always character
Row names	Yes	No
Subsetting	Drops to vector often	Keeps tibble
List columns	Awkward	Native support

Use **tibble** when possible. It's what `read_csv` and all tidyverse functions return.

4 Part 2 — Creating Data

4.1 2.1 From vectors

```
# --- Self-contained: data.frame from vectors ---
p2_df <- data.frame(
  id      = 1:6,
  gender  = c("M", "F", "M", "F", "M", "F"),
  age     = c(25, 30, 22, 28, 35, 27),
  score   = c(78, 85, 90, 72, 88, 95)
)
kable(p2_df)
```

id	gender	age	score
1	M	25	78
2	F	30	85
3	M	22	90
4	F	28	72
5	M	35	88
6	F	27	95

What happens: Each argument becomes a column. All vectors must be the same length (or length 1, which gets recycled).

4.2 2.2 From tibble()

```
# --- Self-contained: tibble from vectors ---
p2_tib <- tibble(
  id      = 1:6,
  gender  = c("M", "F", "M", "F", "M", "F"),
  age     = c(25, 30, 22, 28, 35, 27),
  score   = c(78, 85, 90, 72, 88, 95)
)
kable(p2_tib)
```

id	gender	age	score
1	M	25	78
2	F	30	85
3	M	22	90
4	F	28	72
5	M	35	88
6	F	27	95

Difference from data.frame: strings stay as strings (no silent factor conversion), nice printing, tibble class.

4.3 2.3 From a matrix

```
# --- Self-contained: matrix to data frame ---
p2_mat <- matrix(1:12, nrow = 4, ncol = 3)
```

```
p2_df <- as.data.frame(p2_mat)
names(p2_df) <- c("a", "b", "c")
kable(p2_df)
```

	a	b	c
1	5	9	
2	6	10	
3	7	11	
4	8	12	

When to use: When the data is naturally rectangular and numeric (e.g., a correlation matrix, a count table).

4.4 2.4 From a matrix of counts (exam Q22-3 style)

```
# --- Self-contained: contingency table as matrix ---
p2_tab <- matrix(c(4, 0, 0, 43, 97, 89, 25, 87, 61),
                 nrow = 3, byrow = TRUE)
dimnames(p2_tab) <- list(
  risk = c("low", "medium", "high"),
  group = c("g1", "g2", "g3")
)
kable(p2_tab)
```

	g1	g2	g3
low	4	0	0
medium	43	97	89
high	25	87	61

What happens: `matrix(..., byrow = TRUE)` fills row by row. `dimnames` gives names to rows and columns. This is how you enter a table printed in the exam.

4.5 2.5 From lists (mixed types)

```
# --- Self-contained: list to data frame ---
p2_list <- list(
  id = 1:3,
  name = c("A", "B", "C"),
  val = c(1.1, 2.2, 3.3)
)
p2_df <- as_tibble(p2_list)
kable(p2_df)
```

id	name	val
1	A	1.1
2	B	2.2
3	C	3.3

4.6 2.6 With tribble() (row-wise, easy to read)

```
# --- Self-contained: tribble ---
p2_trib <- tribble(
  ~id, ~gender, ~age, ~score,
  1,   "M",    25,   78,
  2,   "F",    30,   85,
  3,   "M",    22,   90,
  4,   "F",    28,   72
)
kable(p2_trib)
```

id	gender	age	score
1	M	25	78
2	F	30	85
3	M	22	90
4	F	28	72

Use `tribble()` when you want to type the data in by hand and see it aligned like a table.

4.7 2.7 Sequences and repeats

```
# --- Self-contained: sequences ---
p2_seq1 <- 1:10
p2_seq2 <- seq(0, 1, by = 0.25)
p2_seq3 <- seq(0, 1, length.out = 5)
p2_rep1 <- rep(1:3, times = 2)   # 1 2 3 1 2 3
p2_rep2 <- rep(1:3, each = 2)   # 1 1 2 2 3 3
```

```
p2_seq1
```

```
## [1] 1 2 3 4 5 6 7 8 9 10
```

```
p2_seq2
```

```
## [1] 0.00 0.25 0.50 0.75 1.00
```

```
p2_seq3
```

```
## [1] 0.00 0.25 0.50 0.75 1.00
```

```
p2_rep1
```

```
## [1] 1 2 3 1 2 3
```

```
p2_rep2
```

```
## [1] 1 1 2 2 3 3
```

5 Part 3 — Import

5.1 3.1 Excel (.xls, .xlsx)

```
library(readxl)

df <- read_excel("Q1.xls")           # first sheet
df <- read_excel("Q1.xlsx", sheet = "Data") # by name
df <- read_excel("Q1.xls", sheet = 2)   # by index
df <- read_excel("Q1.xls", range = "A1:F72") # specific range
df <- read_excel("Q1.xls", skip = 2)     # skip first 2 rows
df <- read_excel("Q1.xls", col_names = FALSE) # no header row
df <- read_excel("Q1.xls", na = c("", "NA", "?"))
excel_sheets("Q1.xls")                # see sheet names
```

Common issues: - **Header on row 2 or 3:** use `skip = 1` or `skip = 2`. - **Trailing spaces in names:** use `rename_with(trimws)`. - **Wrong type for a column:** fix after reading with `as.numeric()`. - **Missing values coded as text:** use `na = c("", "NA", ".", "?")`.

5.2 3.2 SPSS (.sav)

```
library(haven)
df <- read_sav("data.sav")
df$gender <- as_factor(df$gender) # convert labelled to factor
```

What happens: SPSS files store values with **labels** (e.g., 1 = “Male”, 2 = “Female”). `read_sav` reads numbers; `as_factor` applies the labels. Always convert categorical codes to factors for analysis.

5.3 3.3 SAS / Stata

```
df <- read_sas("data.sas7bdat")
df <- read_dta("data.dta")
df <- read_stata("data.dta")
```

5.4 3.4 Text files

```
df <- read.table("DATA.txt", header = TRUE, sep = "\t") # tab
df <- read.csv("DATA.csv", header = TRUE)               # comma
df <- read.table("DATA.txt", header = TRUE)             # whitespace
df <- read_delim("DATA.txt", delim = ";")              # custom
df <- read_csv("DATA.csv")                             # tidyverse, faster
```

Common issues: - **Wrong delimiter:** try `sep = "\t"`, `sep = ";"`, `sep = "|"`. - **Decimal comma:** use `read_csv2()` for `;`-separated with `,` decimal. - **Encoding issues:** try `locale = locale(encoding = "latin1")`.

5.5 3.5 From clipboard

```
df <- read.table("clipboard", header = TRUE)           # Windows
df <- read.table(pipe("pbpaste"), header = TRUE)       # macOS
df <- read_clip_tbl()                                 # tidyverse, cross-platform
```

5.6 3.6 From a URL

```
df <- read_csv("https://example.com/data.csv")
```

5.7 3.7 After reading — always run these five

```
# --- Self-contained: five inspection commands ---
p3_df <- tibble(
  id      = 1:6,
  gender  = c("M", "F", "M", "F", "M", "F"),
  age     = c(25, 30, 22, 28, 35, 27),
  score   = c(78, 85, 90, 72, 88, 95)
)

dim(p3_df)                                # rows and columns

## [1] 6 4

str(p3_df)                                # structure: types + sample values

## tibble [6 x 4] (S3: tbl_df/tbl/data.frame)
## $ id      : int [1:6] 1 2 3 4 5 6
## $ gender: chr [1:6] "M" "F" "M" "F" ...
## $ age     : num [1:6] 25 30 22 28 35 27
## $ score   : num [1:6] 78 85 90 72 88 95

head(p3_df, 3)                            # first 3 rows

## # A tibble: 3 x 4
##   id gender age score
##   <int> <chr> <dbl> <dbl>
## 1     1 M     25     78
## 2     2 F     30     85
## 3     3 M     22     90

names(p3_df)                              # column names

## [1] "id"      "gender" "age"    "score"

colSums(is.na(p3_df))                     # NA count per column

##   id gender age score
##   0     0   0     0
```

Report template: > “The data has 6 rows and 4 columns: id (integer), gender (character), age (numeric), score (numeric). There are no missing values.”

6 Part 4 — Inspection in Detail

6.1 4.1 Structure

```
# --- Self-contained: structure ---
p4_df <- tibble(
  id      = 1:5,
  gender  = c("M", "F", "M", "F", "M"),
```

```

  age    = c(25, 30, 22, 28, 35),
  score  = c(78, 85, 90, 72, 88)
)

nrow(p4_df)          # 5

## [1] 5
ncol(p4_df)          # 4

## [1] 4
dim(p4_df)           # 5 4

## [1] 5 4
names(p4_df)         # column names

## [1] "id"      "gender" "age"    "score"
str(p4_df)           # types and sample values

## tibble [5 x 4] (S3: tbl_df/tbl/data.frame)
## $ id      : int [1:5] 1 2 3 4 5
## $ gender  : chr [1:5] "M" "F" "M" "F" ...
## $ age     : num [1:5] 25 30 22 28 35
## $ score   : num [1:5] 78 85 90 72 88

glimpse(p4_df)       # tidyverse version of str

## Rows: 5
## Columns: 4
## $ id      <int> 1, 2, 3, 4, 5
## $ gender  <chr> "M", "F", "M", "F", "M"
## $ age     <dbl> 25, 30, 22, 28, 35
## $ score   <dbl> 78, 85, 90, 72, 88

sapply(p4_df, class) # class of each column

##           id           gender           age           score
##  "integer" "character"    "numeric"    "numeric"

```

6.2 4.2 Summary statistics

```

# --- Self-contained: summary ---
p4_df <- tibble(
  id      = 1:5,
  gender  = c("M", "F", "M", "F", "M"),
  age     = c(25, 30, 22, 28, 35),
  score   = c(78, 85, 90, 72, 88)
)

summary(p4_df)

##           id           gender           age           score
##  Min.      :1   Length   :5   Min.      :22   Min.      :72.0
##  1st Qu.:2   N.unique   :2   1st Qu.:25   1st Qu.:78.0
##  Median :3   N.blank    :0   Median :28   Median :85.0

```

```
## Mean      :3   Min.nchar:1   Mean      :28   Mean      :82.6
## 3rd Qu.   :4   Max.nchar:1   3rd Qu.   :30   3rd Qu.   :88.0
## Max.      :5                               Max.      :35   Max.      :90.0
```

Output: For numeric columns: min, Q1, median, mean, Q3, max. For character/factor: counts per level.

6.3 4.3 Unique values

```
# --- Self-contained: unique values ---
p4_df <- tibble(
  gender = c("M", "F", "M", "F", "M"),
  result = c("yes", "no", "yes", "yes", "no")
)
```

```
unique(p4_df$gender)
```

```
## [1] "M" "F"
```

```
table(p4_df$gender)
```

```
##
## F M
## 2 3
```

```
length(unique(p4_df$gender))
```

```
## [1] 2
```

Use this before converting to factor — you want to know every level that exists.

6.4 4.4 Missing values overview

```
# --- Self-contained: NA overview ---
p4_df <- tibble(
  id    = 1:5,
  score = c(78, NA, 90, 72, NA),
  age   = c(25, 30, NA, 28, 35)
)
```

```
sum(is.na(p4_df))           # total NAs
```

```
## [1] 3
```

```
colSums(is.na(p4_df))       # NAs per column
```

```
##   id score  age
##    0     2    1
```

```
rowSums(is.na(p4_df))       # NAs per row
```

```
## [1] 0 1 1 0 1
```

```
mean(is.na(p4_df))          # fraction missing overall
```

```
## [1] 0.2
```

6.5 4.5 Quick views of the top and bottom

```
# --- Self-contained: head and tail ---
```

```
p4_df <- tibble(  
  id    = 1:20,  
  score = rnorm(20)  
)  
head(p4_df)           # first 6
```

```
## # A tibble: 6 x 2  
##       id    score  
##   <int>  <dbl>  
## 1     1 -0.925  
## 2     2 -0.326  
## 3     3 -0.0405  
## 4     4 -0.674  
## 5     5  0.747  
## 6     6 -0.0390
```

```
head(p4_df, 3)        # first 3
```

```
## # A tibble: 3 x 2  
##       id    score  
##   <int>  <dbl>  
## 1     1 -0.925  
## 2     2 -0.326  
## 3     3 -0.0405
```

```
tail(p4_df)           # last 6
```

```
## # A tibble: 6 x 2  
##       id    score  
##   <int>  <dbl>  
## 1    15  1.63  
## 2    16 -1.34  
## 3    17  1.22  
## 4    18  0.611  
## 5    19 -0.664  
## 6    20 -0.0896
```

```
tail(p4_df, 2)        # last 2
```

```
## # A tibble: 2 x 2  
##       id    score  
##   <int>  <dbl>  
## 1    19 -0.664  
## 2    20 -0.0896
```

7 Part 5 — Data Types

7.1 5.1 The seven types you'll meet

Type	class()	Example	Used for
double (numeric)	"numeric"	3.14, -1.5e3	measurements
integer	"integer"	5L, -2L	counts
character	"character"	"male", "A"	text labels
factor	"factor"	group A/B/C	categories
ordered factor	c("ordered", "factor")	low<mid<high	ordinal
logical	"logical"	TRUE, FALSE	flags
Date	"Date"	2024-01-01	calendar dates
POSIXct	c("POSIXct", "POSIXt")	2024-01-01 10:30:00	date-times

7.2 5.2 Numeric

```
# --- Self-contained: numeric ---
p5_a <- 3.14
p5_b <- 1e3      # 1000
p5_c <- -2.5
p5_d <- 5L      # integer (L suffix)

class(p5_a)      # "numeric"

## [1] "numeric"

class(p5_d)      # "integer"

## [1] "integer"

is.numeric(p5_a) # TRUE

## [1] TRUE

is.integer(p5_a) # FALSE (default is double)

## [1] FALSE
```

7.3 5.3 Character

```
# --- Self-contained: character ---
p5_name <- "Parsa"
p5_greeting <- "Hello"
p5_sentence <- 'She said "hi"'

nchar(p5_name)

## [1] 5

toupper(p5_name)

## [1] "PARSA"

paste(p5_greeting, p5_name)

## [1] "Hello Parsa"

paste0(p5_greeting, ", ", p5_name, "!")

## [1] "Hello, Parsa!"
```

7.4 5.4 Logical

```
# --- Self-contained: logical ---
p5_t <- TRUE
p5_f <- FALSE
p5_cmp <- 3 > 2

p5_t & p5_f      # FALSE

## [1] FALSE
p5_t | p5_f      # TRUE

## [1] TRUE
!p5_t            # FALSE

## [1] FALSE
sum(c(TRUE, TRUE, FALSE))  # 2 (TRUE = 1, FALSE = 0)

## [1] 2
```

Trick: `sum(logical)` counts how many are TRUE. `mean(logical)` gives the proportion.

7.5 5.5 Dates

```
# --- Self-contained: dates ---
p5_d1 <- as.Date("2024-01-15")
p5_d2 <- as.Date("15/01/2024", format = "%d/%m/%Y")

class(p5_d1)

## [1] "Date"
format(p5_d1, "%A %d %B %Y")  # "Monday 15 January 2024"

## [1] "Monday 15 January 2024"
p5_d2 - p5_d1                 # Time difference

## Time difference of 0 days
as.numeric(p5_d2 - p5_d1)     # number of days

## [1] 0
```

7.6 5.6 Date-times

```
# --- Self-contained: date-times ---
p5_dt <- as.POSIXct("2024-01-15 10:30:00", tz = "UTC")
class(p5_dt)

## [1] "POSIXct" "POSIXt"
format(p5_dt, "%Y-%m-%d %H:%M")

## [1] "2024-01-15 10:30"
```

7.7 5.7 Checking types

```
# --- Self-contained: type checks ---  
p5_v <- c(1, 2, 3)
```

```
is.numeric(p5_v)
```

```
## [1] TRUE
```

```
is.character(p5_v)
```

```
## [1] FALSE
```

```
is.factor(p5_v)
```

```
## [1] FALSE
```

```
is.logical(p5_v)
```

```
## [1] FALSE
```

```
is.Date(p5_v)
```

```
## [1] FALSE
```

8 Part 6 — Type Conversion

8.1 6.1 Explicit conversion

```
# --- Self-contained: explicit conversion ---  
p6_df <- tibble(  
  a = c("1", "2", "3"),  
  b = c("10", "20", "30"),  
  c = c("M", "F", "M")  
)
```

```
p6_df$a <- as.numeric(p6_df$a)
```

```
p6_df$b <- as.integer(p6_df$b)
```

```
p6_df$c <- as.factor(p6_df$c)
```

```
sapply(p6_df, class)
```

```
##           a           b           c
```

```
## "numeric" "integer" "factor"
```

8.2 6.2 The dangerous conversion

```
# --- Self-contained: coercion warnings ---  
as.numeric(c("1", "2", "three"))
```

```
## [1] 1 2 NA
```

Result: 1 2 NA with a warning. **Always** run `unique()` first to see what values exist.

Rule: If `as.numeric()` gives NA with a warning, the column contains non-numeric characters (spaces, commas, letters). Clean them first:

```
# --- Self-contained: cleaning before conversion ---
p6_raw <- c("1,000", "2,500", "3,200")
p6_clean <- as.numeric(gsub(",", "", p6_raw))
p6_clean
```

```
## [1] 1000 2500 3200
```

8.3 6.3 Character → factor → numeric trap

```
# --- Self-contained: factor-to-numeric trap ---
p6_f <- factor(c("10", "20", "30"))
as.numeric(p6_f) # 1 2 3 (level codes!)
```

```
## [1] 1 2 3
```

```
as.numeric(as.character(p6_f)) # 10 20 30 (correct)
```

```
## [1] 10 20 30
```

Report: “When a factor is coerced to numeric directly, R returns level codes (1, 2, 3). To recover the original numbers, convert to character first.”

8.4 6.4 Character → Date

```
# --- Self-contained: character to Date ---
p6_dates <- c("2024-01-15", "2024-02-20", "2024-03-10")
as.Date(p6_dates) # ISO format
```

```
## [1] "2024-01-15" "2024-02-20" "2024-03-10"
```

```
as.Date("15/01/2024", format = "%d/%m/%Y") # day-first
```

```
## [1] "2024-01-15"
```

```
as.Date("01/15/2024", format = "%m/%d/%Y") # month-first
```

```
## [1] "2024-01-15"
```

8.5 6.5 Logical → numeric

```
# --- Self-contained: logical to numeric ---
p6_lgl <- c(TRUE, FALSE, TRUE)
as.numeric(p6_lgl) # 1 0 1
```

```
## [1] 1 0 1
```

```
as.integer(p6_lgl) # 1 0 1
```

```
## [1] 1 0 1
```

8.6 6.6 Conversion summary table

From	To	Function
character	numeric	<code>as.numeric(x)</code>
character	integer	<code>as.integer(x)</code>
character	factor	<code>factor(x, levels = ...)</code>

From	To	Function
numeric	character	<code>as.character(x)</code>
numeric	factor	<code>factor(x, levels = ...)</code>
factor	character	<code>as.character(x)</code>
factor	numeric (original)	<code>as.numeric(as.character(x))</code>
logical	numeric	<code>as.numeric(x)</code>
character	Date	<code>as.Date(x, format = ...)</code>

9 Part 7 — Factors and Reference Levels

9.1 7.1 Why factors matter

Regression, ANOVA, and mixed models treat factors as **categorical** and create dummy variables. **The first level is the reference** unless you set it otherwise.

```
# --- Self-contained: default factor ordering ---
p7_v <- c("control", "treat1", "treat2", "control", "treat1")
p7_f <- factor(p7_v)
levels(p7_f) # alphabetical: "control" "treat1" "treat2"

## [1] "control" "treat1" "treat2"
```

9.2 7.2 Explicit levels (setting the reference)

```
# --- Self-contained: explicit levels ---
p7_df <- tibble(
  id = 1:5,
  group = c("control", "treat1", "treat2", "control", "treat1")
)

p7_df$group <- factor(p7_df$group,
  levels = c("control", "treat1", "treat2"))
levels(p7_df$group)

## [1] "control" "treat1" "treat2"
```

control is now the reference. Any regression using group will have coefficients interpreted as “difference from control”.

9.3 7.3 Change the reference later

```
# --- Self-contained: relevel ---
p7_df <- tibble(group = factor(c("control", "treat1", "treat2", "control")))
p7_df$group <- relevel(p7_df$group, ref = "treat1")
levels(p7_df$group)

## [1] "treat1" "control" "treat2"
```

9.4 7.4 Check dummy coding

```
# --- Self-contained: contrasts ---
p7_df <- tibble(
  group = factor(c("control", "treat1", "treat2", "control", "treat1"),
    levels = c("control", "treat1", "treat2"))
)
contrasts(p7_df$group)
```

```
##          treat1 treat2
## control         0      0
## treat1          1      0
## treat2          0      1
```

Output: Two columns (treat1, treat2), each is 0/1 for that level. The reference control is all zeros.

9.5 7.5 Numeric-coded groups (Q1 style)

If groups are coded 1, 2, 3 but 3 should be the reference:

```
# --- Self-contained: numeric-coded to factor with custom reference ---
p7_df <- tibble(
  id      = 1:6,
  treatment = c(1, 2, 3, 1, 2, 3)
)

p7_df$treatment <- factor(p7_df$treatment, levels = c(3, 1, 2))
levels(p7_df$treatment) # "3" "1" "2"
```

```
## [1] "3" "1" "2"
```

9.6 7.6 Ordered factors (ordinal data)

```
# --- Self-contained: ordered factor ---
p7_df <- tibble(
  education = c("low", "medium", "high", "medium", "low")
)

p7_df$education <- factor(p7_df$education,
  levels = c("low", "medium", "high"),
  ordered = TRUE)

levels(p7_df$education)
```

```
## [1] "low"      "medium"   "high"
```

```
is.ordered(p7_df$education)
```

```
## [1] TRUE
```

Use **ordered factors** when the levels have a natural order (low < medium < high). This changes the default contrasts to polynomial, but the levels are the same.

9.7 7.7 Manual dummies

```
# --- Self-contained: manual dummy ---
p7_df <- tibble(
  id      = 1:5,
  gender = c("M", "F", "M", "F", "M")
)
```

```
)

p7_df$is_male <- ifelse(p7_df$gender == "M", 1, 0)
p7_df$is_female <- ifelse(p7_df$gender == "F", 1, 0)
kable(p7_df)
```

id	gender	is_male	is_female
1	M	1	0
2	F	0	1
3	M	1	0
4	F	0	1
5	M	1	0

Use **manual dummies** when you want explicit control over the design matrix.

9.8 7.8 Recode factor levels

```
# --- Self-contained: fct_recode ---
library(forcats)
p7_f <- factor(c("M", "F", "M", "F"))
p7_f2 <- fct_recode(p7_f, "Male" = "M", "Female" = "F")
levels(p7_f2)
```

```
## [1] "Female" "Male"
```

9.9 7.9 Collapse rare levels

```
# --- Self-contained: fct_lump ---
library(forcats)
p7_f <- factor(c("A", "A", "A", "B", "B", "C", "D"))
fct_lump(p7_f, n = 2) # keep top 2, lump rest as "Other"
```

```
## [1] A      A      A      B      B      Other Other
## Levels: A B Other
```

Report template: > “The factor group has three levels: control, treat1, treat2. I set control as the reference because it is the natural baseline. Regression coefficients for treat1 and treat2 will then represent the difference in mean outcome vs control.”

10 Part 8 — Missing Values

10.1 8.1 Detecting

```
# --- Self-contained: NA detection ---
p8_df <- tibble(
  id    = 1:5,
  score = c(78, NA, 90, 72, NA),
  age   = c(25, 30, NA, 28, 35)
)
```

```

is.na(p8_df)                                # matrix of TRUE/FALSE

##      id score  age
## [1,] FALSE FALSE FALSE
## [2,] FALSE  TRUE FALSE
## [3,] FALSE FALSE  TRUE
## [4,] FALSE FALSE FALSE
## [5,] FALSE  TRUE FALSE

sum(is.na(p8_df))                          # total

## [1] 3

colSums(is.na(p8_df))                      # per column

##      id score  age
##      0      2      1

rowSums(is.na(p8_df))                     # per row

## [1] 0 1 1 0 1

anyNA(p8_df)                              # any NA?

## [1] TRUE

mean(is.na(p8_df))                       # fraction missing overall

## [1] 0.2

which(is.na(p8_df$score))                 # row indices with NA

## [1] 2 5

```

10.2 8.2 Removing rows with NA

```

# --- Self-contained: remove NAs ---
p8_df <- tibble(
  id    = 1:5,
  score = c(78, NA, 90, 72, NA)
)

p8_clean1 <- na.omit(p8_df)
p8_clean2 <- p8_df %>% filter(!is.na(score))
p8_clean3 <- tidyr::drop_na(p8_df)

nrow(p8_clean1)    # 3

## [1] 3

nrow(p8_clean2)    # 3

## [1] 3

nrow(p8_clean3)    # 3

## [1] 3

```

What happens: `na.omit()` removes any row with at least one NA. `filter(!is.na(x))` removes only rows where `x` is NA. Choose based on which columns matter.

10.3 8.3 Replacing NA

```
# --- Self-contained: replace NAs ---
p8_df <- tibble(
  id    = 1:5,
  score = c(78, NA, 90, 72, NA)
)

# Replace with mean
p8_mean <- mean(p8_df$score, na.rm = TRUE)
p8_df$score[is.na(p8_df$score)] <- p8_mean
kable(p8_df)
```

id	score
1	78
2	80
3	90
4	72
5	80

```
# --- Self-contained: replace with constant ---
p8_df <- tibble(id = 1:5, score = c(78, NA, 90, 72, NA))
p8_df$score[is.na(p8_df$score)] <- 0
kable(p8_df)
```

id	score
1	78
2	0
3	90
4	72
5	0

```
# --- Self-contained: replace with median ---
p8_df <- tibble(id = 1:5, score = c(78, NA, 90, 72, NA))
p8_df$score[is.na(p8_df$score)] <- median(p8_df$score, na.rm = TRUE)
kable(p8_df)
```

id	score
1	78
2	78
3	90
4	72
5	78

10.4 8.4 Filling (LOCF — last observation carried forward)

```
# --- Self-contained: LOCF ---
p8_df <- tibble(
  time = 1:5,
```

```

  score = c(78, NA, NA, 72, NA)
)
p8_filled <- p8_df %>% tidyr::fill(score, .direction = "down")
kable(p8_filled)

```

time	score
1	78
2	78
3	78
4	72
5	72

When to use: Time series where the value is expected to persist until the next observed value.

10.5 8.5 Interpolation

```

# --- Self-contained: linear interpolation ---
p8_df <- tibble(
  time = 1:5,
  score = c(78, NA, NA, 72, 75)
)
p8_df$score <- approx(p8_df$time, p8_df$score, xout = p8_df$time)$y
kable(p8_df, digits = 2)

```

time	score
1	78
2	76
3	74
4	72
5	75

When to use: Continuous measurements over time.

10.6 8.6 na.rm = TRUE in functions

```

# --- Self-contained: na.rm ---
p8_v <- c(78, NA, 90, 72, NA)

mean(p8_v)          # NA

## [1] NA
mean(p8_v, na.rm = TRUE)  # 80

## [1] 80
sum(p8_v, na.rm = TRUE)

## [1] 240
sd(p8_v, na.rm = TRUE)

```

```
## [1] 9.165151
```

```
median(p8_v, na.rm = TRUE)
```

```
## [1] 78
```

Always use `na.rm = TRUE` in summary functions, unless you actually want NA propagation.

10.7 8.7 NA vs NaN vs NULL vs Inf

Value	Meaning	Behaves as
NA	missing	propagates; check with <code>is.na()</code>
NaN	not a number (0/0)	is also NA; check with <code>is.nan()</code>
NULL	empty	length 0; disappears in vectors
Inf	infinity	real value; check with <code>is.infinite()</code>

```
# --- Self-contained: special values ---
```

```
c(1, NA, 3)      # length 3
```

```
## [1] 1 NA 3
```

```
c(1, NULL, 3)    # length 2
```

```
## [1] 1 3
```

```
c(1, NaN, 3)     # length 3
```

```
## [1] 1 NaN 3
```

```
c(1, Inf, 3)     # length 3
```

```
## [1] 1 Inf 3
```

```
is.na(NaN)       # TRUE
```

```
## [1] TRUE
```

```
is.nan(NaN)      # TRUE
```

```
## [1] TRUE
```

```
is.na(NA)        # TRUE
```

```
## [1] TRUE
```

```
is.nan(NA)       # FALSE
```

```
## [1] FALSE
```

10.8 8.8 Report template

If you removed: > “The score column has 2 missing values (40%). I removed the affected rows using `drop_na()`, leaving 3 complete observations. The remaining data has no missing values.”

If you filled: > “The score column has 2 missing values. I imputed them with the column mean (80.0), so all subsequent analyses use a complete dataset of 5 rows. Sensitivity to this imputation should be considered.”

Warning: If >5% of the data is missing, note it as a limitation. If missingness is systematic (e.g., all men missing), it biases the analysis.

11 Part 9 — Column Operations

11.1 9.1 Select columns

```
# --- Self-contained: select ---
p9_df <- tibble(
  id      = 1:5,
  gender  = c("M", "F", "M", "F", "M"),
  age     = c(25, 30, 22, 28, 35),
  score   = c(78, 85, 90, 72, 88),
  q1      = c(1, 2, 3, 4, 5),
  q2      = c(6, 7, 8, 9, 10)
)

p9_df %>% select(id, score)                                # by name

## # A tibble: 5 x 2
##       id score
##   <int> <dbl>
## 1     1    78
## 2     2    85
## 3     3    90
## 4     4    72
## 5     5    88

p9_df %>% select(1:3)                                      # by position

## # A tibble: 5 x 3
##       id gender  age
##   <int> <chr>  <dbl>
## 1     1  M      25
## 2     2  F      30
## 3     3  M      22
## 4     4  F      28
## 5     5  M      35

p9_df %>% select(-id)                                      # drop one

## # A tibble: 5 x 5
##   gender  age score  q1  q2
##   <chr>  <dbl> <dbl> <dbl> <dbl>
## 1 M      25    78    1    6
## 2 F      30    85    2    7
## 3 M      22    90    3    8
## 4 F      28    72    4    9
## 5 M      35    88    5   10

p9_df %>% select(starts_with("q"))                        # by prefix

## # A tibble: 5 x 2
##       q1  q2
##   <dbl> <dbl>
## 1     1    6
## 2     2    7
## 3     3    8
## 4     4    9
```

```
## 5      5      10
p9_df %>% select(ends_with("e"))           # by suffix

## # A tibble: 5 x 2
##   age score
##   <dbl> <dbl>
## 1    25    78
## 2    30    85
## 3    22    90
## 4    28    72
## 5    35    88

p9_df %>% select(contains("co"))           # by substring

## # A tibble: 5 x 1
##   score
##   <dbl>
## 1    78
## 2    85
## 3    90
## 4    72
## 5    88

p9_df %>% select(where(is.numeric))       # by type

## # A tibble: 5 x 5
##   id   age score   q1   q2
##   <int> <dbl> <dbl> <dbl> <dbl>
## 1     1    25    78     1     6
## 2     2    30    85     2     7
## 3     3    22    90     3     8
## 4     4    28    72     4     9
## 5     5    35    88     5    10
```

11.2 9.2 Rename columns

```
# --- Self-contained: rename ---
p9_df <- tibble(id = 1:3, score = c(10, 20, 30), age = c(25, 30, 35))

p9_df %>% rename(score_final = score, age_years = age)

## # A tibble: 3 x 3
##   id score_final age_years
##   <int>      <dbl>      <dbl>
## 1     1         10         25
## 2     2         20         30
## 3     3         30         35
```

11.3 9.3 Rename all columns

```
# --- Self-contained: rename_with ---
p9_df <- tibble(Score = 1:3, Age = c(25, 30, 35))

p9_df %>% rename_with(tolower)           # all lowercase
```

```
## # A tibble: 3 x 2
##   score age
##   <int> <dbl>
## 1     1    25
## 2     2    30
## 3     3    35

p9_df %>% rename_with(~ paste0("v_", .x))      # prefix

## # A tibble: 3 x 2
##   v_Score v_Age
##   <int> <dbl>
## 1     1    25
## 2     2    30
## 3     3    35
```

11.4 9.4 Reorder columns

```
# --- Self-contained: relocate ---
p9_df <- tibble(id = 1:3, score = c(10, 20, 30), age = c(25, 30, 35))

p9_df %>% relocate(score, .after = id)

## # A tibble: 3 x 3
##       id score age
##   <int> <dbl> <dbl>
## 1     1    10  25
## 2     2    20  30
## 3     3    30  35

p9_df %>% relocate(score, .before = id)

## # A tibble: 3 x 3
##   score id age
##   <dbl> <int> <dbl>
## 1    10     1  25
## 2    20     2  30
## 3    30     3  35

p9_df %>% relocate(where(is.numeric), .after = id)

## # A tibble: 3 x 3
##       id score age
##   <int> <dbl> <dbl>
## 1     1    10  25
## 2     2    20  30
## 3     3    30  35
```

11.5 9.5 Extract a column as a vector

```
# --- Self-contained: pull ---
p9_df <- tibble(id = 1:3, score = c(10, 20, 30))
p9_df %>% pull(score)      # vector

## [1] 10 20 30
```

```
p9_df %>% pull(1) # first column
```

```
## [1] 1 2 3
```

```
p9_df$score # same
```

```
## [1] 10 20 30
```

11.6 9.6 Base R alternatives

```
# --- Self-contained: base R column ops ---
```

```
# --- Self-contained: base R column ops ---
```

```
p9_df <- tibble(id = 1:3, score = c(10, 20, 30), age = c(25, 30, 35))
```

```
p9_df[, c("id", "score")] # select columns by name
```

```
## # A tibble: 3 x 2
```

```
##   id score
```

```
##   <int> <dbl>
```

```
## 1     1    10
```

```
## 2     2    20
```

```
## 3     3    30
```

```
p9_df[, -1] # drop the first column
```

```
## # A tibble: 3 x 2
```

```
##   score age
```

```
##   <dbl> <dbl>
```

```
## 1    10   25
```

```
## 2    20   30
```

```
## 3    30   35
```

```
p9_df[, 1:2] # select by position
```

```
## # A tibble: 3 x 2
```

```
##   id score
```

```
##   <int> <dbl>
```

```
## 1     1    10
```

```
## 2     2    20
```

```
## 3     3    30
```

```
p9_df[, c("score", "id", "age")] # reorder columns
```

```
## # A tibble: 3 x 3
```

```
##   score id age
```

```
##   <dbl> <int> <dbl>
```

```
## 1    10     1  25
```

```
## 2    20     2  30
```

```
## 3    30     3  35
```

```
names(p9_df)[2] <- "score_final" # rename the 2nd column (do this LAST) # reorder
```

12 Part 10 — Row Operations

12.1 10.1 Filter rows

```
# --- Self-contained: filter ---
p10_df <- tibble(
  id      = 1:6,
  gender  = c("M", "F", "M", "F", "M", "F"),
  age     = c(25, 30, 22, 28, 35, 27),
  score   = c(78, 85, 90, 72, 88, 95)
)

p10_df %>% filter(gender == "M") # equals

## # A tibble: 3 x 4
##       id gender   age score
##   <int> <chr>  <dbl> <dbl>
## 1     1  M      25     78
## 2     3  M      22     90
## 3     5  M      35     88

p10_df %>% filter(age > 25) # greater than

## # A tibble: 4 x 4
##       id gender   age score
##   <int> <chr>  <dbl> <dbl>
## 1     2  F      30     85
## 2     4  F      28     72
## 3     5  M      35     88
## 4     6  F      27     95

p10_df %>% filter(age > 25, score > 80) # AND

## # A tibble: 3 x 4
##       id gender   age score
##   <int> <chr>  <dbl> <dbl>
## 1     2  F      30     85
## 2     5  M      35     88
## 3     6  F      27     95

p10_df %>% filter(gender == "M" | age > 30) # OR

## # A tibble: 3 x 4
##       id gender   age score
##   <int> <chr>  <dbl> <dbl>
## 1     1  M      25     78
## 2     3  M      22     90
## 3     5  M      35     88

p10_df %>% filter(gender %in% c("M", "F")) # in

## # A tibble: 6 x 4
##       id gender   age score
##   <int> <chr>  <dbl> <dbl>
## 1     1  M      25     78
## 2     2  F      30     85
## 3     3  M      22     90
```

```
## 4      4 F      28    72
## 5      5 M      35    88
## 6      6 F      27    95
```

```
p10_df %>% filter(between(age, 20, 30))           # range
```

```
## # A tibble: 5 x 4
##       id gender  age score
##   <int> <chr>  <dbl> <dbl>
## 1     1 M      25    78
## 2     2 F      30    85
## 3     3 M      22    90
## 4     4 F      28    72
## 5     6 F      27    95
```

```
p10_df %>% filter(!is.na(score))                 # not NA
```

```
## # A tibble: 6 x 4
##       id gender  age score
##   <int> <chr>  <dbl> <dbl>
## 1     1 M      25    78
## 2     2 F      30    85
## 3     3 M      22    90
## 4     4 F      28    72
## 5     5 M      35    88
## 6     6 F      27    95
```

```
p10_df %>% filter(str_detect(gender, "M"))       # regex
```

```
## # A tibble: 3 x 4
##       id gender  age score
##   <int> <chr>  <dbl> <dbl>
## 1     1 M      25    78
## 2     3 M      22    90
## 3     5 M      35    88
```

Common mistakes: - `filter(gender = "M")` — should be `==`. - `filter(age > 25 & score > 80)` — works, but `,` is faster. - Forgetting `%in%` for multiple matches.

12.2 10.2 Slice rows by position

```
# --- Self-contained: slice ---
```

```
p10_df <- tibble(id = 1:10, score = 1:10)
```

```
p10_df %>% slice(1:3)                           # first 3 rows
```

```
## # A tibble: 3 x 2
##       id score
##   <int> <int>
## 1     1     1
## 2     2     2
## 3     3     3
```

```
p10_df %>% slice(c(1, 5, 10))                   # specific rows
```

```
## # A tibble: 3 x 2
##       id score
```

```
##      <int> <int>
## 1      1      1
## 2      5      5
## 3     10     10

p10_df %>% slice_head(n = 3)      # first 3

## # A tibble: 3 x 2
##       id score
##   <int> <int>
## 1     1     1
## 2     2     2
## 3     3     3

p10_df %>% slice_tail(n = 3)      # last 3

## # A tibble: 3 x 2
##       id score
##   <int> <int>
## 1     8     8
## 2     9     9
## 3    10    10

p10_df %>% slice_max(score, n = 3) # top 3 by score

## # A tibble: 3 x 2
##       id score
##   <int> <int>
## 1    10    10
## 2     9     9
## 3     8     8

p10_df %>% slice_min(score, n = 3) # bottom 3

## # A tibble: 3 x 2
##       id score
##   <int> <int>
## 1     1     1
## 2     2     2
## 3     3     3

p10_df %>% slice_sample(n = 3)    # random 3

## # A tibble: 3 x 2
##       id score
##   <int> <int>
## 1     5     5
## 2     3     3
## 3     8     8
```

12.3 10.3 Distinct (unique) rows

```
# --- Self-contained: distinct ---
p10_df <- tibble(
  id      = c(1, 1, 2, 3, 3),
  group   = c("A", "A", "B", "A", "B")
)
```

```
p10_df %>% distinct() # unique combinations
```

```
## # A tibble: 4 x 2
##   id group
##   <dbl> <chr>
## 1     1 A
## 2     2 B
## 3     3 A
## 4     3 B
```

```
p10_df %>% distinct(id) # unique id values
```

```
## # A tibble: 3 x 1
##   id
##   <dbl>
## 1     1
## 2     2
## 3     3
```

```
p10_df %>% distinct(id, .keep_all = TRUE) # first row per id
```

```
## # A tibble: 3 x 2
##   id group
##   <dbl> <chr>
## 1     1 A
## 2     2 B
## 3     3 A
```

12.4 10.4 Sort rows

```
# --- Self-contained: arrange ---
```

```
p10_df <- tibble(
  id      = 1:6,
  gender  = c("M", "F", "M", "F", "M", "F"),
  score   = c(78, 85, 90, 72, 88, 95)
)
```

```
p10_df %>% arrange(score) # ascending
```

```
## # A tibble: 6 x 3
##   id gender score
##   <int> <chr> <dbl>
## 1     4 F      72
## 2     1 M      78
## 3     2 F      85
## 4     5 M      88
## 5     3 M      90
## 6     6 F      95
```

```
p10_df %>% arrange(desc(score)) # descending
```

```
## # A tibble: 6 x 3
##   id gender score
##   <int> <chr> <dbl>
## 1     6 F      95
```

```
## 2      3 M      90
## 3      5 M      88
## 4      2 F      85
## 5      1 M      78
## 6      4 F      72
```

```
p10_df %>% arrange(gender, desc(score)) # multi-key
```

```
## # A tibble: 6 x 3
##       id gender score
##   <int> <chr> <dbl>
## 1     6 F      95
## 2     2 F      85
## 3     4 F      72
## 4     3 M      90
## 5     5 M      88
## 6     1 M      78
```

12.5 10.5 Base R alternatives

```
# --- Self-contained: base R row ops ---
```

```
p10_df <- tibble(id = 1:5, score = c(78, 85, 90, 72, 88))
```

```
p10_df[p10_df$score > 80, ] # filter
```

```
## # A tibble: 3 x 2
##       id score
##   <int> <dbl>
## 1     2  85
## 2     3  90
## 3     5  88
```

```
subset(p10_df, score > 80) # filter (simpler)
```

```
## # A tibble: 3 x 2
##       id score
##   <int> <dbl>
## 1     2  85
## 2     3  90
## 3     5  88
```

```
p10_df[order(p10_df$score), ] # ascending
```

```
## # A tibble: 5 x 2
##       id score
##   <int> <dbl>
## 1     4  72
## 2     1  78
## 3     2  85
## 4     5  88
## 5     3  90
```

```
p10_df[order(-p10_df$score), ] # descending
```

```
## # A tibble: 5 x 2
##       id score
##   <int> <dbl>
```

```
## 1      3      90
## 2      5      88
## 3      2      85
## 4      1      78
## 5      4      72

p10_df[!duplicated(p10_df$id), ]      # distinct

## # A tibble: 5 x 2
##       id score
##   <int> <dbl>
## 1     1     78
## 2     2     85
## 3     3     90
## 4     4     72
## 5     5     88
```

12.6 10.6 Row counts before and after

```
# --- Self-contained: verify row counts ---
p10_df <- tibble(id = 1:100, score = rnorm(100))

n_before <- nrow(p10_df)
p10_filtered <- p10_df %>% filter(score > 0)
n_after <- nrow(p10_filtered)

c(before = n_before, after = n_after)
```

```
## before after
##   100    52
```

Always check: If `n_after == 0`, you filtered out everything. If `n_after == n_before`, the filter did nothing. Both are red flags.

13 Part 11 — Creating and Modifying Columns

13.1 11.1 Mutate — add or modify columns

```
# --- Self-contained: mutate ---
p11_df <- tibble(
  id    = 1:5,
  score = c(78, 85, 90, 72, 88),
  age   = c(25, 30, 22, 28, 35)
)

p11_df <- p11_df %>% mutate(
  log_score = log(score),
  score2    = score^2,
  age_c     = age - mean(age),
  age_z     = (age - mean(age)) / sd(age),
  result    = ifelse(score >= 80, "pass", "fail")
)

kable(p11_df, digits = 3)
```

id	score	age	log_score	score2	age_c	age_z	result
1	78	25	4.357	6084	-3	-0.606	fail
2	85	30	4.443	7225	2	0.404	pass
3	90	22	4.500	8100	-6	-1.212	pass
4	72	28	4.277	5184	0	0.000	fail
5	88	35	4.477	7744	7	1.414	pass

What happens: `mutate` adds new columns. It can reference columns just created in the same call.

13.2 11.2 ifelse vs if_else

```
# --- Self-contained: ifelse vs if_else ---
p11_df <- tibble(score = c(78, 85, 90, NA))

# Base ifelse - returns NA when score is NA
p11_df$base <- ifelse(p11_df$score >= 80, "pass", "fail")

# dplyr if_else - strict, but you control the NA
p11_df$dplyr <- if_else(p11_df$score >= 80, "pass", "fail", missing = "unknown")

kable(p11_df)
```

score	base	dplyr
78	fail	fail
85	pass	pass
90	pass	pass
NA	NA	unknown

Use `if_else` when you want type-checking and explicit NA handling.

13.3 11.3 case_when (multiple conditions)

```
# --- Self-contained: case_when ---
p11_df <- tibble(score = c(95, 85, 75, 65, 55))

p11_df <- p11_df %>% mutate(
  grade = case_when(
    score >= 90 ~ "A",
    score >= 80 ~ "B",
    score >= 70 ~ "C",
    score >= 60 ~ "D",
    TRUE      ~ "F"
  )
)
kable(p11_df)
```

score	grade
95	A
85	B

score	grade
75	C
65	D
55	F

Why `case_when`: Cleaner than nested `ifelse`. The `TRUE ~ ...` at the end catches everything else.

13.4 11.4 cut (bin numeric into categories)

```
# --- Self-contained: cut ---
p11_df <- tibble(age = c(15, 22, 28, 35, 45, 60, 72))

p11_df$age_cat <- cut(p11_df$age,
  breaks = c(0, 25, 40, 60, 100),
  labels = c("young", "middle", "senior", "elder"),
  right = FALSE)

kable(p11_df)
```

age	age_cat
15	young
22	young
28	middle
35	middle
45	senior
60	elder
72	elder

Arguments: - `breaks`: boundary values. - `labels`: names for intervals. `Length = length(breaks) - 1`. - `right = FALSE`: intervals `[a, b)`. `right = TRUE` (default): `(a, b]`. - `include.lowest = TRUE`: include the lowest boundary.

13.5 11.5 Rownames

```
# --- Self-contained: rownames ---
p11_df <- tibble(id = 1:3, score = c(10, 20, 30))

# Convert rownames to a column
p11_mtcars <- head(mtcars)
p11_mtcars$car <- rownames(p11_mtcars)
head(p11_mtcars[, c("car", "mpg")], 2)

##              car mpg
## Mazda RX4      21
## Mazda RX4 Wag  21

# Or use tibble::rownames_to_column
p11_mtcars2 <- tibble::rownames_to_column(head(mtcars), var = "car")
head(p11_mtcars2[, c("car", "mpg")], 2)

##              car mpg
## 1      Mazda RX4  21
## 2 Mazda RX4 Wag  21
```

13.6 11.6 Transmute (keep only new columns)

```
# --- Self-contained: transmute ---
p11_df <- tibble(x = 1:3, y = 4:6)
p11_df %>% transmute(z = x + y, w = x * y)
```

```
## # A tibble: 3 x 2
##       z     w
##   <int> <int>
## 1     5     4
## 2     7    10
## 3     9    18
```

13.7 11.7 across — apply to many columns

```
# --- Self-contained: across ---
p11_df <- tibble(a = c(1, 2, 3), b = c(4, 5, 6), c = c("x", "y", "z"))

p11_df %>% mutate(across(where(is.numeric), ~ .x * 2))
```

```
## # A tibble: 3 x 3
##       a     b c
##   <dbl> <dbl> <chr>
## 1     2     8 x
## 2     4    10 y
## 3     6    12 z
```

```
p11_df %>% mutate(across(where(is.numeric), ~ .x - mean(.x), .names = "{.col}_c"))
```

```
## # A tibble: 3 x 5
##       a     b c     a_c b_c
##   <dbl> <dbl> <chr> <dbl> <dbl>
## 1     1     4 x     -1    -1
## 2     2     5 y      0     0
## 3     3     6 z      1     1
```

Report template: > “I created a BMI column as weight / height², and categorized age into four groups (young, middle, senior, elder). I also standardized age to have mean 0 and SD 1.”

14 Part 12 — Grouped Operations

14.1 12.1 group_by + summarise

```
# --- Self-contained: grouped summary ---
p12_df <- tibble(
  gender = c("M", "F", "M", "F", "M", "F"),
  age    = c(25, 30, 22, 28, 35, 27),
  score  = c(78, 85, 90, 72, 88, 95)
)

p12_df %>%
  group_by(gender) %>%
  summarise(
```

```

n      = n(),
mean_s = mean(score),
sd_s   = sd(score),
median_s = median(score),
min_s  = min(score),
max_s  = max(score),
.groups = "drop"
) %>%
kable(digits = 3)

```

gender	n	mean_s	sd_s	median_s	min_s	max_s
F	3	84.000	11.533	85	72	95
M	3	85.333	6.429	88	78	90

What happens: `group_by` splits the data by the grouping variable. `summarise` computes one row per group.

14.2 12.2 Summarise multiple columns

```

# --- Self-contained: summarise across ---
p12_df <- tibble(
  gender = c("M", "F", "M", "F", "M", "F"),
  age    = c(25, 30, 22, 28, 35, 27),
  score  = c(78, 85, 90, 72, 88, 95)
)

p12_df %>%
  group_by(gender) %>%
  summarise(across(c(age, score),
                    list(mean = mean, sd = sd),
                    .names = "{.col}_{.fn}"),
            .groups = "drop") %>%
  kable(digits = 3)

```

gender	age_mean	age_sd	score_mean	score_sd
F	28.333	1.528	84.000	11.533
M	27.333	6.807	85.333	6.429

14.3 12.3 Count per group

```

# --- Self-contained: count ---
p12_df <- tibble(gender = c("M", "F", "M", "F", "M", "F"))

p12_df %>% count(gender)

## # A tibble: 2 x 2
##   gender     n
##   <chr> <int>
## 1 F         3
## 2 M         3

```

```
p12_df %>% count(gender, sort = TRUE)
```

```
## # A tibble: 2 x 2
##   gender      n
##   <chr>   <int>
## 1 F         3
## 2 M         3
```

```
p12_df %>% count(gender, name = "n_obs")
```

```
## # A tibble: 2 x 2
##   gender n_obs
##   <chr>   <int>
## 1 F         3
## 2 M         3
```

14.4 12.4 Grouped mutate (add group-level summary)

```
# --- Self-contained: grouped mutate ---
p12_df <- tibble(
  gender = c("M", "F", "M", "F", "M", "F"),
  score = c(78, 85, 90, 72, 88, 95)
)

p12_df %>%
  group_by(gender) %>%
  mutate(mean_score = mean(score),
         diff_from_mean = score - mean_score) %>%
  ungroup() %>%
  kable(digits = 3)
```

gender	score	mean_score	diff_from_mean
M	78	85.333	-7.333
F	85	84.000	1.000
M	90	85.333	4.667
F	72	84.000	-12.000
M	88	85.333	2.667
F	95	84.000	11.000

Use case: Standardizing within groups, computing deviations, ranking within groups.

14.5 12.5 Grouped filter (keep rows satisfying group condition)

```
# --- Self-contained: grouped filter ---
p12_df <- tibble(
  gender = c("M", "F", "M", "F", "M", "F"),
  score = c(78, 85, 90, 72, 88, 95)
)

# Keep rows where score is above the group mean
p12_df %>%
  group_by(gender) %>%
```

```
filter(score > mean(score)) %>%
ungroup()
```

```
## # A tibble: 4 x 2
##   gender score
##   <chr>   <dbl>
## 1 F      85
## 2 M      90
## 3 M      88
## 4 F      95
```

14.6 12.6 Ungroup

```
# --- Self-contained: ungroup ---
p12_df <- tibble(gender = c("M", "F"), score = c(80, 90))

p12_df %>% group_by(gender) %>% summarise(n = n(), .groups = "drop")
```

```
## # A tibble: 2 x 2
##   gender      n
##   <chr>   <int>
## 1 F         1
## 2 M         1
```

```
# or explicitly
p12_df %>% group_by(gender) %>% summarise(n = n()) %>% ungroup()
```

```
## # A tibble: 2 x 2
##   gender      n
##   <chr>   <int>
## 1 F         1
## 2 M         1
```

Always `.groups = "drop"` or `ungroup()` at the end, otherwise downstream functions use the grouping silently.

14.7 12.7 Base R alternatives

```
# --- Self-contained: base R group summary ---
p12_df <- tibble(
  gender = c("M", "F", "M", "F", "M", "F"),
  score = c(78, 85, 90, 72, 88, 95)
)

tapply(p12_df$score, p12_df$gender, mean)
```

```
##           F           M
## 84.00000 85.33333
```

```
aggregate(score ~ gender, data = p12_df, FUN = mean)
```

```
##   gender      score
## 1      F 84.00000
## 2      M 85.33333
```

```
aggregate(cbind(score, score) ~ gender, data = p12_df, FUN = mean)
```

```
##   gender   score   score
## 1      F 84.00000 84.00000
## 2      M 85.33333 85.33333
```

14.8 12.8 Report template

“Mean score is 85.3 for men ($n = 3$, $SD = 6.4$) and 84.0 for women ($n = 3$, $SD = 11.5$). The group means differ by 1.3 points, with wider variability in the female group.”

15 Part 13 — Window Functions

Window functions return a value **per row** but depend on a group or order.

15.1 13.1 Lag and lead

```
# --- Self-contained: lag and lead ---
p13_df <- tibble(
  year = 2020:2024,
  value = c(10, 12, 15, 14, 18)
)

p13_df %>%
  mutate(
    prev = lag(value),
    next_v = lead(value),
    change = value - prev
  ) %>%
  kable()
```

year	value	prev	next_v	change
2020	10	NA	12	NA
2021	12	10	15	2
2022	15	12	14	3
2023	14	15	18	-1
2024	18	14	NA	4

Use for: time series, computing changes, growth rates.

15.2 13.2 Cumulative functions

```
# --- Self-contained: cumsum etc ---
p13_df <- tibble(x = c(1, 2, 3, 4, 5))

p13_df %>%
  mutate(
    cum_sum = cumsum(x),
    cum_max = cummax(x),
    cum_min = cummin(x),
```

```

  cum_prod = cumprod(x)
) %>%
kable()

```

x	cum_sum	cum_max	cum_min	cum_prod
1	1	1	1	1
2	3	2	1	2
3	6	3	1	6
4	10	4	1	24
5	15	5	1	120

15.3 13.3 Ranking

```

# --- Self-contained: ranking ---
p13_df <- tibble(score = c(90, 75, 85, 75, 95))

p13_df %>%
  mutate(
    row_num = row_number(),
    rank    = rank(score),
    rank_desc = rank(desc(score)),
    min_rank = min_rank(score),
    dense_rank = dense_rank(score),
    pct_rank = percent_rank(score)
  ) %>%
  kable(digits = 3)

```

score	row_num	rank	rank_desc	min_rank	dense_rank	pct_rank
90	1	4.0	2.0	4	3	0.75
75	2	1.5	4.5	1	1	0.00
85	3	3.0	3.0	3	2	0.50
75	4	1.5	4.5	1	1	0.00
95	5	5.0	1.0	5	4	1.00

Function	Behavior with ties
row_number()	always unique; ties broken by order
min_rank()	ties get the same rank; next rank skips
dense_rank()	ties get the same rank; next rank not skipped
percent_rank()	scaled to [0, 1]

15.4 13.4 n() and n_distinct()

```

# --- Self-contained: n and n_distinct ---
p13_df <- tibble(
  group = c("A", "A", "B", "B", "B"),
  id     = c(1, 2, 3, 4, 5)
)

```

```
p13_df %>%
  group_by(group) %>%
  summarise(n_rows = n(), n_ids = n_distinct(id), .groups = "drop")
```

```
## # A tibble: 2 x 3
##   group n_rows n_ids
##   <chr> <int> <int>
## 1 A         2     2
## 2 B         3     3
```

15.5 13.5 first, last, nth

```
# --- Self-contained: first/last/nth ---
p13_df <- tibble(
  group = c("A", "A", "A", "B", "B"),
  value = c(10, 20, 30, 40, 50)
)
```

```
p13_df %>%
  group_by(group) %>%
  summarise(
    first_v = first(value),
    last_v = last(value),
    second_v = nth(value, 2),
    .groups = "drop"
  )
```

```
## # A tibble: 2 x 4
##   group first_v last_v second_v
##   <chr>   <dbl> <dbl>   <dbl>
## 1 A         10     30       20
## 2 B         40     50       50
```

16 Part 14 — Reshaping

16.1 14.1 Wide to long

```
# --- Self-contained: pivot_longer ---
p14_wide <- tibble(
  id = 1:3,
  y0 = c(10, 12, 14),
  y1 = c(11, 13, 15),
  y2 = c(12, 14, 16)
)

p14_long <- p14_wide %>%
  pivot_longer(
    cols = starts_with("y"),
    names_to = "time",
    values_to = "score"
  ) %>%
```

```
mutate(time = as.numeric(gsub("y", "", time)))

kable(p14_long)
```

id	time	score
1	0	10
1	1	11
1	2	12
2	0	12
2	1	13
2	2	14
3	0	14
3	1	15
3	2	16

What happens: Each row of `p14_wide` becomes 3 rows in `p14_long`. Columns `y0`, `y1`, `y2` become a single `score` column, with `time` identifying which one.

16.2 14.2 Long to wide

```
# --- Self-contained: pivot_wider ---
p14_long <- tibble(
  id    = rep(1:3, each = 3),
  time  = rep(0:2, 3),
  score = c(10, 11, 12, 12, 13, 14, 14, 15, 16)
)

p14_wide <- p14_long %>%
  pivot_wider(
    names_from = time,
    values_from = score,
    names_prefix = "y"
  )
kable(p14_wide)
```

id	y0	y1	y2
1	10	11	12
2	12	13	14
3	14	15	16

16.3 14.3 Separate one column into two

```
# --- Self-contained: separate ---
p14_df <- tibble(code = c("A_1", "B_2", "C_3"))

p14_df %>%
  separate(code, into = c("letter", "number"), sep = "_") %>%
  kable()
```

letter	number
A	1
B	2
C	3

16.4 14.4 Unite two columns into one

```
# --- Self-contained: unite ---
p14_df <- tibble(letter = c("A", "B"), number = 1:2)

p14_df %>%
  unite("code", letter, number, sep = "-") %>%
  kable()
```

code
A-1
B-2

16.5 14.5 Complete (fill in missing combinations)

```
# --- Self-contained: complete ---
p14_df <- tibble(
  year = c(2020, 2021, 2023),
  val = c(1, 2, 4)
)

p14_df %>% tidyr::complete(year = 2020:2023) %>% kable()
```

year	val
2020	1
2021	2
2022	NA
2023	4

Use case: Filling in missing years, months, or group combinations.

16.6 14.6 Long vs wide: which to use

Task	Format
Time series plot with multiple series	long
Regression with one row per subject	wide
ANOVA / repeated measures	long
Correlation matrix	wide
Summary table for report	wide

16.7 14.7 Report template

“The data was originally in wide format (y0, y1, y2 columns). I converted it to long format so that each row represents one observation (id, time, score). This is the format required for mixed models and repeated-measures ANOVA.”

17 Part 15 — Combining Data Frames

17.1 15.1 Stack rows (union)

```
# --- Self-contained: bind_rows ---
p15_a <- tibble(x = 1:3, y = c("a", "b", "c"))
p15_b <- tibble(x = 4:6, y = c("d", "e", "f"))

bind_rows(p15_a, p15_b)
```

```
## # A tibble: 6 x 2
##       x y
##   <int> <chr>
## 1     1 a
## 2     2 b
## 3     3 c
## 4     4 d
## 5     5 e
## 6     6 f
```

```
rbind(p15_a, p15_b)
```

```
## # A tibble: 6 x 2
##       x y
##   <int> <chr>
## 1     1 a
## 2     2 b
## 3     3 c
## 4     4 d
## 5     5 e
## 6     6 f
```

When columns differ: `bind_rows` fills missing columns with NA.

```
# --- Self-contained: bind_rows with different columns ---
p15_a <- tibble(x = 1:2)
p15_b <- tibble(x = 3:4, y = c("p", "q"))

bind_rows(p15_a, p15_b)
```

```
## # A tibble: 4 x 2
##       x y
##   <int> <chr>
## 1     1 <NA>
## 2     2 <NA>
## 3     3 p
## 4     4 q
```

17.2 15.2 Stack columns (side-by-side)

```
# --- Self-contained: bind_cols ---
```

```
p15_a <- tibble(x = 1:3)
```

```
p15_b <- tibble(y = 4:6)
```

```
bind_cols(p15_a, p15_b)
```

```
## # A tibble: 3 x 2
```

```
##       x     y
```

```
##   <int> <int>
```

```
## 1     1     4
```

```
## 2     2     5
```

```
## 3     3     6
```

```
cbind(p15_a, p15_b)
```

```
##   x y
```

```
## 1 1 4
```

```
## 2 2 5
```

```
## 3 3 6
```

Warning: bind_cols matches by position, not by name. Rows must align.

17.3 15.3 The six joins

```
# --- Self-contained: all joins ---
```

```
p15_left  <- tibble(id = 1:3, x = c("a", "b", "c"))
```

```
p15_right <- tibble(id = 2:4, y = c("P", "Q", "R"))
```

```
# Inner: only matching ids
```

```
inner_join(p15_left, p15_right, by = "id")
```

```
## # A tibble: 2 x 3
```

```
##       id x     y
```

```
##   <int> <chr> <chr>
```

```
## 1     2 b     P
```

```
## 2     3 c     Q
```

```
# Left: all rows from left, matched from right
```

```
left_join(p15_left, p15_right, by = "id")
```

```
## # A tibble: 3 x 3
```

```
##       id x     y
```

```
##   <int> <chr> <chr>
```

```
## 1     1 a     <NA>
```

```
## 2     2 b     P
```

```
## 3     3 c     Q
```

```
# Right: all rows from right, matched from left
```

```
right_join(p15_left, p15_right, by = "id")
```

```
## # A tibble: 3 x 3
```

```
##       id x     y
```

```
##   <int> <chr> <chr>
```

```
## 1     2 b     P
```

```
## 2     3 c     Q
```

```
## 3      4 <NA> R
# Full: all rows from both
full_join(p15_left, p15_right, by = "id")
```

```
## # A tibble: 4 x 3
##   id x     y
##   <int> <chr> <chr>
## 1     1 a     <NA>
## 2     2 b     P
## 3     3 c     Q
## 4     4 <NA> R
```

```
# Anti: rows in left NOT in right
anti_join(p15_left, p15_right, by = "id")
```

```
## # A tibble: 1 x 2
##   id x
##   <int> <chr>
## 1     1 a
```

```
# Semi: rows in left that ARE in right (left columns only)
semi_join(p15_left, p15_right, by = "id")
```

```
## # A tibble: 2 x 2
##   id x
##   <int> <chr>
## 1     2 b
## 2     3 c
```

17.4 15.4 Joining by different key names

```
# --- Self-contained: join with different key names ---
p15_a <- tibble(id_a = 1:3, x = c("a", "b", "c"))
p15_b <- tibble(id_b = 2:4, y = c("P", "Q", "R"))

inner_join(p15_a, p15_b, by = c("id_a" = "id_b"))
```

```
## # A tibble: 2 x 3
##   id_a x     y
##   <int> <chr> <chr>
## 1     2 b     P
## 2     3 c     Q
```

17.5 15.5 Join summary

Join	Keeps
inner_join	only matching rows
left_join	all rows from left
right_join	all rows from right
full_join	all rows from both
anti_join	rows in left NOT in right
semi_join	rows in left that match right

17.6 15.6 Report template

“I merged the demographics table with the outcomes table using `left_join` by `id`. The result has 200 rows (one per subject), with 3 subjects having missing outcome data (they appear with NA).”

18 Part 16 — Frequency and Contingency Tables

18.1 16.1 One-way frequency

```
# --- Self-contained: one-way table ---
p16_df <- tibble(
  gender = c("M", "F", "M", "F", "M", "F", "M", "F")
)

table(p16_df$gender)
```

```
##
## F M
## 4 4
```

```
prop.table(table(p16_df$gender))
```

```
##
##   F   M
## 0.5 0.5
```

```
round(prop.table(table(p16_df$gender)) * 100, 1)
```

```
##
##   F   M
## 50 50
```

18.2 16.2 Two-way contingency table

```
# --- Self-contained: two-way table ---
p16_df <- tibble(
  gender = c("M", "F", "M", "F", "M", "F", "M", "F"),
  result = c("yes", "no", "yes", "yes", "no", "no", "yes", "yes")
)

tab <- table(p16_df$gender, p16_df$result)
addmargins(tab)
```

```
##
##      no yes Sum
##   F    2  2   4
##   M    1  3   4
## Sum    3  5   8
```

18.3 16.3 Proportions

```

# --- Self-contained: proportion tables ---
p16_df <- tibble(
  gender = c("M", "F", "M", "F", "M", "F", "M", "F"),
  result = c("yes", "no", "yes", "yes", "no", "no", "yes", "yes")
)
tab <- table(p16_df$gender, p16_df$result)

prop.table(tab)                                # overall %

##
##      no   yes
##  F 0.250 0.250
##  M 0.125 0.375

prop.table(tab, margin = 1)                    # row % (within each gender)

##
##      no   yes
##  F 0.50 0.50
##  M 0.25 0.75

prop.table(tab, margin = 2)                    # column % (within each result)

##
##      no      yes
##  F 0.6666667 0.4000000
##  M 0.3333333 0.6000000

round(prop.table(tab, margin = 1) * 100, 1)

##
##      no yes
##  F 50  50
##  M 25  75

```

Rule of thumb: For a 2×2 table, use **row percentages** (margin = 1) if gender is the predictor and result is the outcome.

18.4 16.4 Chi-square test

```

# --- Self-contained: chi-square ---
p16_df <- tibble(
  gender = c("M", "F", "M", "F", "M", "F", "M", "F"),
  result = c("yes", "no", "yes", "yes", "no", "no", "yes", "yes")
)
tab <- table(p16_df$gender, p16_df$result)

chisq.test(tab)

##
##  Pearson's Chi-squared test with Yates' continuity correction
##
## data:  tab
## X-squared = 0, df = 1, p-value = 1

fisher.test(tab)    # for small expected counts

```

```
##
## Fisher's Exact Test for Count Data
##
## data:  tab
## p-value = 1
## alternative hypothesis: true odds ratio is not equal to 1
## 95 percent confidence interval:
##    0.07880583 234.45085003
## sample estimates:
## odds ratio
##    2.601683
```

Interpretation: - H_0 : no association between the two variables. - $p < 0.05$: variables are associated. - Check expected counts: if any < 5 , use Fisher's exact test.

18.5 16.5 CrossTable (gmodels package)

```
library(gmodels)
CrossTable(p16_df$gender, p16_df$result, prop.r = TRUE, prop.c = TRUE)
```

Useful because it prints counts and all proportions in one call.

18.6 16.6 Report template

“The 2-way table of gender and result shows that 66.7% of men and 50.0% of women had a positive result. A chi-square test gives $\chi^2(1) = 0.10$, $p = 0.75$, indicating no significant association between gender and result.”

19 Part 17 — Descriptive Statistics by Group

19.1 17.1 Standard summary table

```
# --- Self-contained: summary by group ---
p17_df <- tibble(
  group = c("A", "A", "A", "B", "B", "B", "C", "C", "C"),
  score = c(78, 85, 90, 72, 88, 95, 60, 75, 82)
)

p17_df %>%
  group_by(group) %>%
  summarise(
    n      = n(),
    mean   = mean(score),
    sd     = sd(score),
    median = median(score),
    min    = min(score),
    max    = max(score),
    .groups = "drop"
  ) %>%
  kable(digits = 3)
```

group	n	mean	sd	median	min	max
A	3	84.333	6.028	85	78	90
B	3	85.000	11.790	88	72	95
C	3	72.333	11.240	75	60	82

19.2 17.2 Custom summary with SE and CI

--- Self-contained: summary with SE and CI ---

```
p17_df <- tibble(
  group = c("A", "A", "A", "B", "B", "B"),
  score = c(78, 85, 90, 72, 88, 95)
)
```

```
p17_df %>%
  group_by(group) %>%
  summarise(
    n = n(),
    mean = mean(score),
    sd = sd(score),
    se = sd / sqrt(n),
    ci_lo = mean - qt(0.975, n - 1) * se,
    ci_hi = mean + qt(0.975, n - 1) * se,
    .groups = "drop"
  ) %>%
  kable(digits = 3)
```

group	n	mean	sd	se	ci_lo	ci_hi
A	3	84.333	6.028	3.480	69.360	99.307
B	3	85.000	11.790	6.807	55.712	114.288

19.3 17.3 Summary by multiple groupings

--- Self-contained: two-way grouping ---

```
p17_df <- tibble(
  gender = c("M", "M", "F", "F", "M", "F"),
  treat = c("A", "B", "A", "B", "A", "B"),
  score = c(78, 85, 90, 72, 88, 95)
)
```

```
p17_df %>%
  group_by(gender, treat) %>%
  summarise(n = n(), mean = mean(score), .groups = "drop") %>%
  kable(digits = 3)
```

gender	treat	n	mean
F	A	1	90.0
F	B	2	83.5
M	A	2	83.0
M	B	1	85.0

19.4 17.4 Formatting for a report

```
# --- Self-contained: formatted summary strings ---
p17_df <- tibble(
  group = c("A", "A", "B", "B"),
  score = c(80, 90, 70, 85)
)

p17_df %>%
  group_by(group) %>%
  summarise(
    m_sd = sprintf("%.1f (%.1f)", mean(score), sd(score)),
    .groups = "drop"
  ) %>%
  kable()
```

group	m_sd
A	85.0 (7.1)
B	77.5 (10.6)

Report style: “Mean (SD)” in one column, common in medical/psych papers.

19.5 17.5 Base R alternatives

```
# --- Self-contained: base R summaries ---
p17_df <- tibble(
  group = c("A", "A", "A", "B", "B", "B"),
  score = c(78, 85, 90, 72, 88, 95)
)

tapply(p17_df$score, p17_df$group, mean)

##           A           B
## 84.33333 85.00000

aggregate(score ~ group, data = p17_df, FUN = function(x) c(mean = mean(x), sd = sd(x)))

##   group score.mean score.sd
## 1     A 84.333333 6.027714
## 2     B 85.000000 11.789826
```

20 Part 18 — Sampling and Splitting

20.1 18.1 Simple random sample

```
# --- Self-contained: simple random sample ---
set.seed(1801)
p18_df <- tibble(id = 1:100, y = rnorm(100))

p18_sample <- p18_df %>% sample_n(10)
p18_sample <- p18_df %>% slice_sample(n = 10)
```

```
p18_sample <- p18_df %>% slice_sample(prop = 0.1) # 10%
nrow(p18_sample)
```

```
## [1] 10
```

20.2 18.2 Train/test split

```
# --- Self-contained: train/test split ---
set.seed(1802)
p18_df <- tibble(id = 1:100, y = rnorm(100))

p18_n      <- nrow(p18_df)
p18_idx    <- sample(1:p18_n, size = floor(0.8 * p18_n))
p18_train  <- p18_df[p18_idx, ]
p18_test   <- p18_df[-p18_idx, ]

nrow(p18_train); nrow(p18_test)
```

```
## [1] 80
```

```
## [1] 20
```

Report: “I split the data 80/20 into training (80 rows) and test (20 rows) sets.”

20.3 18.3 Stratified sample (preserve group proportions)

```
# --- Self-contained: stratified sample ---
set.seed(1803)
p18_df <- tibble(
  gender = rep(c("M", "F"), each = 50),
  y      = rnorm(100)
)

p18_strat <- p18_df %>%
  group_by(gender) %>%
  slice_sample(prop = 0.2) %>%
  ungroup()
nrow(p18_strat)
```

```
## [1] 20
```

```
table(p18_strat$gender)
```

```
##
```

```
## F M
```

```
## 10 10
```

20.4 18.4 K-fold cross-validation indices

```
# --- Self-contained: K-fold indices ---
set.seed(1804)
p18_n <- 100
p18_K <- 5
p18_folds <- sample(rep(1:p18_K, length.out = p18_n))
```

```
for (k in 1:p18_K) {
  cat("Fold", k, ":", sum(p18_folds == k), "rows\n")
}
```

```
## Fold 1 : 20 rows
## Fold 2 : 20 rows
## Fold 3 : 20 rows
## Fold 4 : 20 rows
## Fold 5 : 20 rows
```

20.5 18.5 Report template

“I used `set.seed(42)` before sampling for reproducibility. The 80/20 split gives 80 training and 20 test observations. Stratified by gender, each set has the same male:female ratio as the full data.”

21 Part 19 — Outliers

21.1 19.1 IQR rule

```
# --- Self-contained: IQR outlier detection ---
p19_df <- tibble(y = c(10, 12, 11, 13, 12, 100, 11, 12, 13, 10))

Q1 <- quantile(p19_df$y, 0.25)
Q3 <- quantile(p19_df$y, 0.75)
IQR_y <- Q3 - Q1
lower <- Q1 - 1.5 * IQR_y
upper <- Q3 + 1.5 * IQR_y

p19_df$is_outlier <- p19_df$y < lower | p19_df$y > upper
kable(p19_df)
```

y	is_outlier
10	FALSE
12	FALSE
11	FALSE
13	FALSE
12	FALSE
100	TRUE
11	FALSE
12	FALSE
13	FALSE
10	FALSE

Rule: Outliers are values below $Q_1 - 1.5 \cdot IQR$ or above $Q_3 + 1.5 \cdot IQR$.

21.2 19.2 Standardized residuals

```
# --- Self-contained: Z-score outliers ---
p19_df <- tibble(y = c(10, 12, 11, 13, 12, 100, 11, 12, 13, 10))
```

```
p19_z <- (p19_df$y - mean(p19_df$y)) / sd(p19_df$y)
which(abs(p19_z) > 3)
```

```
## integer(0)
```

Rule: $|z| > 3$ is a common threshold.

21.3 19.3 Remove outliers

```
# --- Self-contained: remove outliers ---
p19_df <- tibble(y = c(10, 12, 11, 13, 12, 100, 11, 12, 13, 10))

Q1 <- quantile(p19_df$y, 0.25)
Q3 <- quantile(p19_df$y, 0.75)
IQR_y <- Q3 - Q1

p19_clean <- p19_df %>%
  filter(y >= Q1 - 1.5 * IQR_y, y <= Q3 + 1.5 * IQR_y)

nrow(p19_df); nrow(p19_clean)

## [1] 10
## [1] 9
```

21.4 19.4 Winsorize (cap at percentiles)

```
# --- Self-contained: winsorize ---
p19_df <- tibble(y = c(10, 12, 11, 13, 12, 100, 11, 12, 13, 10))

p19_df$y_win <- pmin(pmax(p19_df$y, quantile(p19_df$y, 0.05)),
  quantile(p19_df$y, 0.95))
kable(p19_df)
```

y	y_win
10	10.00
12	12.00
11	11.00
13	13.00
12	12.00
100	60.85
11	11.00
12	12.00
13	13.00
10	10.00

Winsorizing replaces extreme values with the 5th and 95th percentiles. Less drastic than removing.

21.5 19.5 Report template

“Using the $1.5 \times \text{IQR}$ rule, I identified 1 outlier ($y = 100$). Removing it reduces the mean from 20.4 to 12.2 and the SD from 27.5 to 1.1. If the outlier is a data-entry error, remove it; otherwise consider winsorizing or reporting results with and without it.”

Caution: Never remove outliers silently. Always report both analyses.

22 Part 20 — Strings

22.1 20.1 Basic operations

```
# --- Self-contained: string basics ---  
p20_s <- " Hello, World! "
```

```
nchar(p20_s)
```

```
## [1] 17
```

```
toupper(p20_s)
```

```
## [1] " HELLO, WORLD! "
```

```
tolower(p20_s)
```

```
## [1] " hello, world! "
```

```
trimws(p20_s)
```

```
## [1] "Hello, World!"
```

```
substr(p20_s, 3, 7)
```

```
## [1] "Hello"
```

```
strsplit("a,b,c", ",")[[1]]
```

```
## [1] "a" "b" "c"
```

```
paste("x", "y", "z")
```

```
## [1] "x y z"
```

```
paste0("x", "y", "z")
```

```
## [1] "xyz"
```

22.2 20.2 Detect, extract, replace

```
# --- Self-contained: string pattern matching ---  
p20_v <- c("apple", "banana", "cherry", "apricot")
```

```
grep("ap", p20_v) # indices
```

```
## [1] 1 4
```

```
grep("ap", p20_v, value = TRUE) # values
```

```
## [1] "apple" "apricot"
```

```
grepl("ap", p20_v) # logical
```

```
## [1] TRUE FALSE FALSE TRUE
```

```
sub("a", "A", p20_v)           # replace first
```

```
## [1] "Apple" "bAnana" "cherry" "Apricot"
```

```
gsub("a", "A", p20_v)         # replace all
```

```
## [1] "Apple" "bAnAnA" "cherry" "Apricot"
```

22.3 20.3 stringr (tidyverse)

```
# --- Self-contained: stringr ---
```

```
library(stringr)
```

```
p20_v <- c("apple", "banana", "cherry", "apricot")
```

```
str_detect(p20_v, "an")       # logical
```

```
## [1] FALSE TRUE FALSE FALSE
```

```
str_replace(p20_v, "a", "A")
```

```
## [1] "Apple" "bAnana" "cherry" "Apricot"
```

```
str_replace_all(p20_v, "a", "A")
```

```
## [1] "Apple" "bAnAnA" "cherry" "Apricot"
```

```
str_sub(p20_v, 1, 3)          # first 3 chars
```

```
## [1] "app" "ban" "che" "apr"
```

```
str_length(p20_v)
```

```
## [1] 5 6 6 7
```

```
str_to_upper(p20_v)
```

```
## [1] "APPLE" "BANANA" "CHERRY" "APRICOT"
```

```
str_trim(" hi ")
```

```
## [1] "hi"
```

```
str_pad("hi", 5, side = "left")
```

```
## [1] "   hi"
```

```
str_split("a,b,c", ",")[[1]]
```

```
## [1] "a" "b" "c"
```

22.4 20.4 Common cleaning tasks

```
# --- Self-contained: cleaning strings ---
```

```
p20_raw <- c("M", "m", "M ", "Male", " F", "f", "FEMALE")
```

```
# Normalize
```

```
p20_clean <- p20_raw %>%
```

```
  str_trim() %>%
```

```
  str_to_upper() %>%
```

```
  str_sub(1, 1) # first letter only
```

```
p20_clean
```

```
## [1] "M" "M" "M" "M" "F" "F" "F"
```

22.5 20.5 Extract with regex

```
# --- Self-contained: regex extraction ---  
p20_codes <- c("A-123", "B-456", "C-789")  
str_extract(p20_codes, "[A-Z]") # letter
```

```
## [1] "A" "B" "C"
```

```
str_extract(p20_codes, "\\d+") # digits
```

```
## [1] "123" "456" "789"
```

```
str_match(p20_codes, "([A-Z])-(\\d+)")
```

```
##      [,1]      [,2] [,3]  
## [1,] "A-123" "A"  "123"  
## [2,] "B-456" "B"  "456"  
## [3,] "C-789" "C"  "789"
```

22.6 20.6 Report template

“The gender column contained inconsistent labels (‘M’, ‘m’, ‘Male’, ‘F’, ‘FEMALE’). I normalized by trimming whitespace, converting to uppercase, and taking the first letter, yielding two clean levels: M and F.”

23 Part 21 — Dates and Times

23.1 21.1 Create dates

```
# --- Self-contained: create dates ---  
p21_d1 <- as.Date("2024-01-15")  
p21_d2 <- as.Date("15/01/2024", format = "%d/%m/%Y")  
p21_dt <- as.POSIXct("2024-01-15 10:30:00", tz = "UTC")  
p21_d1; p21_d2; p21_dt
```

```
## [1] "2024-01-15"
```

```
## [1] "2024-01-15"
```

```
## [1] "2024-01-15 10:30:00 UTC"
```

23.2 21.2 Components

```
# --- Self-contained: date components ---  
p21_d <- as.Date("2024-01-15")  
  
format(p21_d, "%Y") # "2024"
```

```
## [1] "2024"
```

```

format(p21_d, "%m")      # "01"

## [1] "01"
format(p21_d, "%d")      # "15"

## [1] "15"
format(p21_d, "%A")      # "Monday"

## [1] "Monday"
format(p21_d, "%B")      # "January"

## [1] "January"
weekdays(p21_d)

## [1] "Monday"
months(p21_d)

## [1] "January"
quarters(p21_d)

## [1] "Q1"

```

23.3 21.3 Arithmetic

```

# --- Self-contained: date arithmetic ---
p21_d1 <- as.Date("2024-01-15")
p21_d2 <- as.Date("2024-02-15")

p21_d1 + 7                # one week later

## [1] "2024-01-22"
p21_d2 - p21_d1           # Time difference of 31 days

## Time difference of 31 days
as.numeric(p21_d2 - p21_d1)

## [1] 31

```

23.4 21.4 Compute age from birth date

```

# --- Self-contained: age from DOB ---
p21_dob <- as.Date(c("1990-05-10", "1985-03-22", "2000-11-15"))
p21_ref <- as.Date("2024-01-01")
p21_age <- as.numeric(p21_ref - p21_dob) / 365.25
round(p21_age, 1)

## [1] 33.6 38.8 23.1

```

23.5 21.5 lubridate (easier)

```
# --- Self-contained: lubridate ---
library(lubridate)
p21_d <- ymd("2024-01-15")
p21_d

## [1] "2024-01-15"
year(p21_d)

## [1] 2024
month(p21_d)

## [1] 1
day(p21_d)

## [1] 15
wday(p21_d, label = TRUE)

## [1] Mon
## Levels: Sun < Mon < Tue < Wed < Thu < Fri < Sat
p21_d + weeks(2)

## [1] "2024-01-29"
p21_d %m+% months(1) # month arithmetic with rollover

## [1] "2024-02-15"
```

23.6 21.6 Common formats

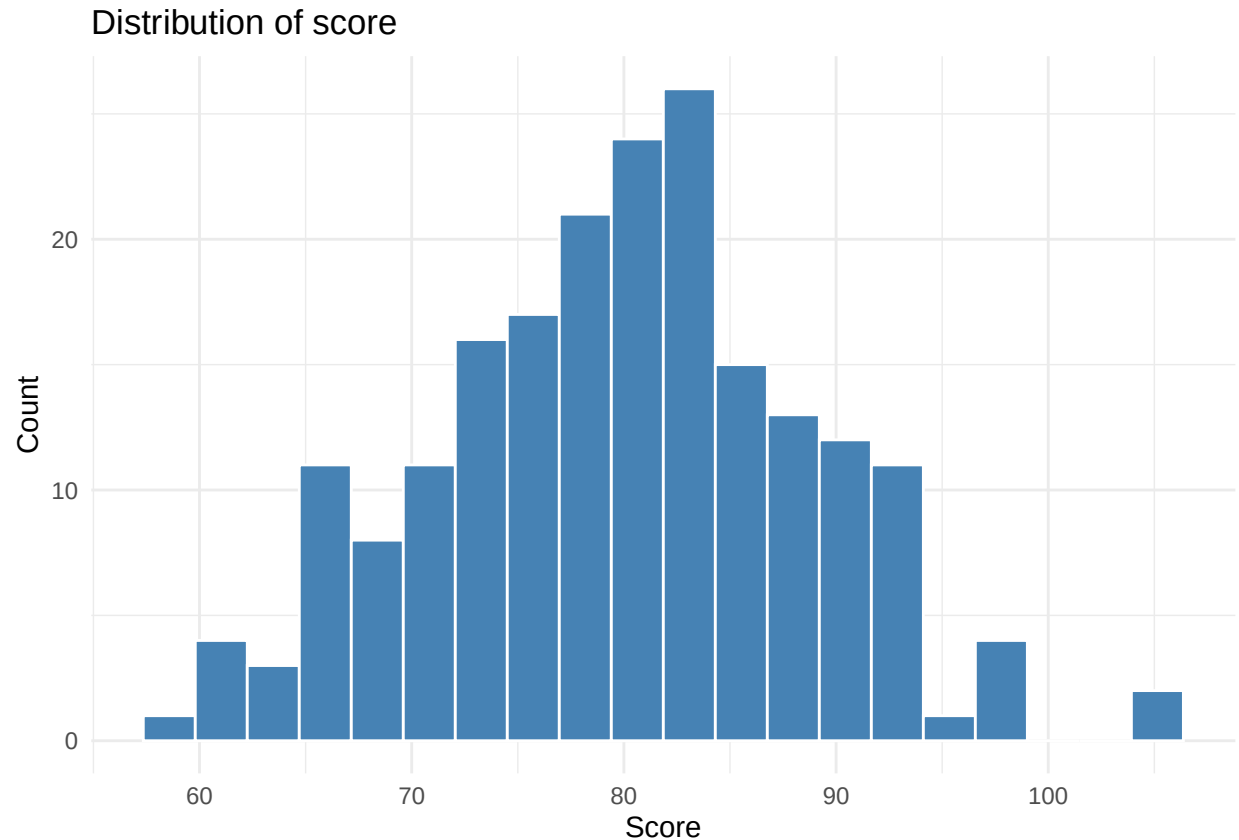
Format	Meaning	Example
%Y	4-digit year	2024
%y	2-digit year	24
%m	month (01–12)	01
%d	day (01–31)	15
%H	hour (00–23)	10
%M	minute	30
%S	second	00
%A	weekday (full)	Monday
%B	month (full)	January

24 Part 22 — Plots for Exploration

24.1 22.1 Histogram

```
# --- Self-contained: histogram ---
set.seed(2201)
p22_df <- tibble(score = rnorm(200, 80, 10))
```

```
ggplot(p22_df, aes(score)) +
  geom_histogram(bins = 20, fill = "steelblue", color = "white") +
  labs(title = "Distribution of score", x = "Score", y = "Count") +
  theme_minimal()
```

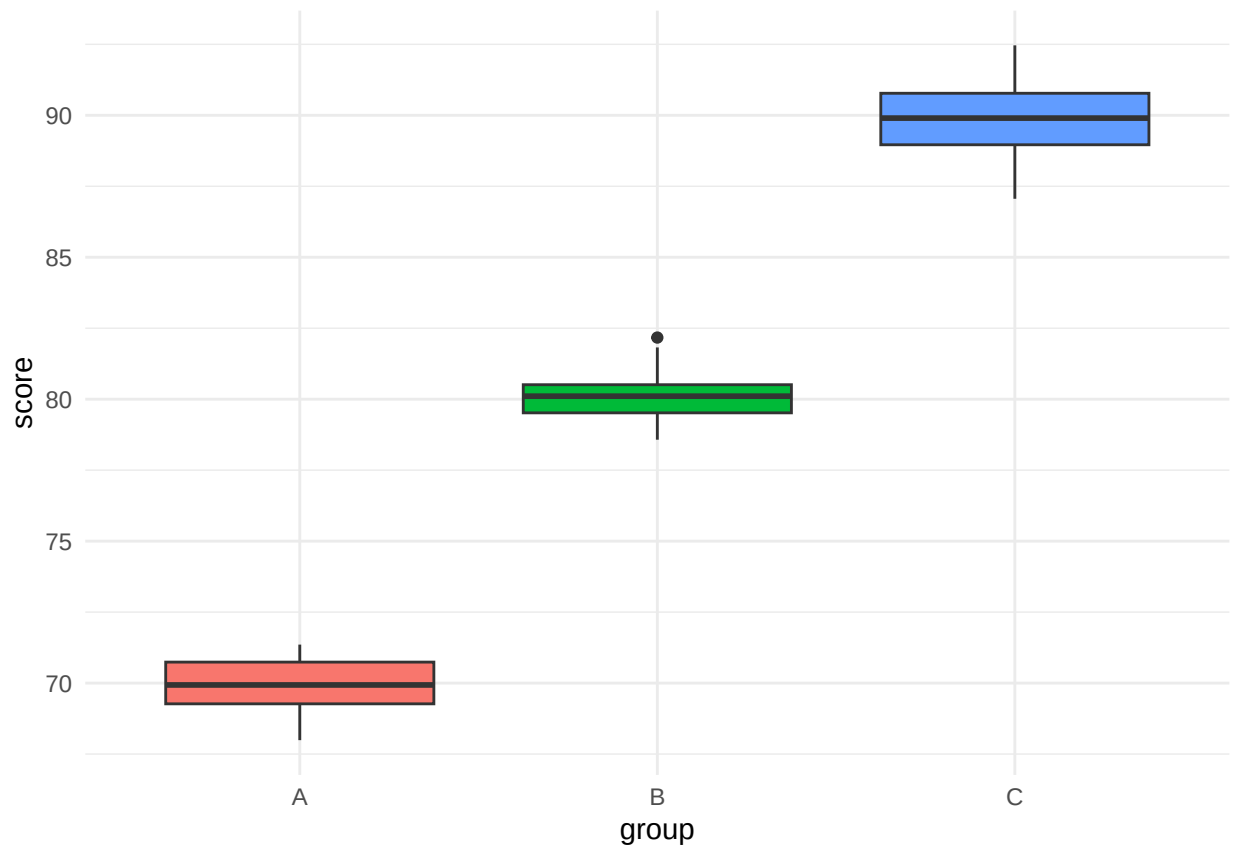


What to look for: shape (symmetric? skewed?), outliers, gaps, multiple modes.

24.2 22.2 Boxplot by group

```
# --- Self-contained: boxplot ---
set.seed(2202)
p22_df <- tibble(
  group = rep(c("A", "B", "C"), each = 30),
  score = c(rnorm(30, 70), rnorm(30, 80), rnorm(30, 90))
)

ggplot(p22_df, aes(group, score, fill = group)) +
  geom_boxplot() +
  theme_minimal() +
  theme(legend.position = "none")
```

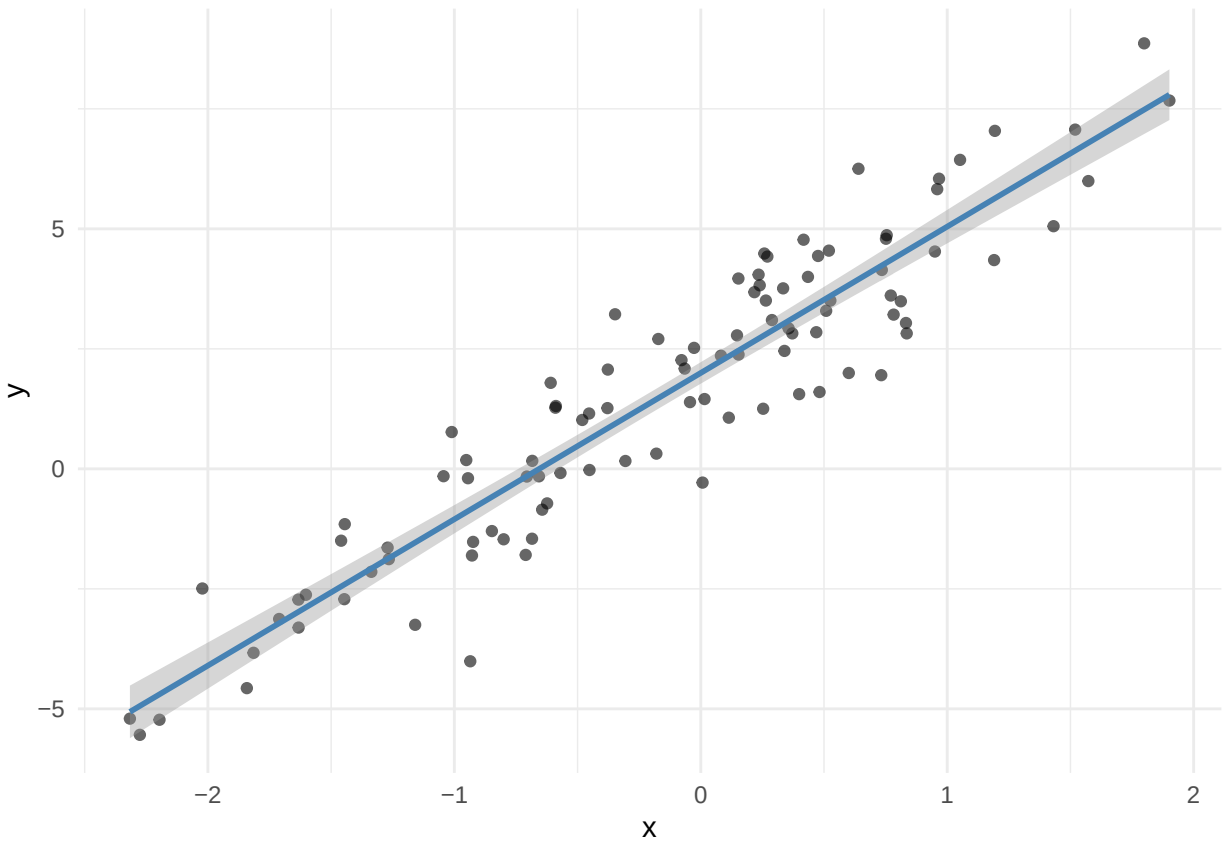


What to look for: differences in medians, spread, outliers, skewness.

24.3 22.3 Scatterplot

```
# --- Self-contained: scatterplot ---
set.seed(2203)
p22_df <- tibble(x = rnorm(100), y = 2 + 3 * x + rnorm(100))

ggplot(p22_df, aes(x, y)) +
  geom_point(alpha = 0.6) +
  geom_smooth(method = "lm", se = TRUE, color = "steelblue") +
  theme_minimal()
```

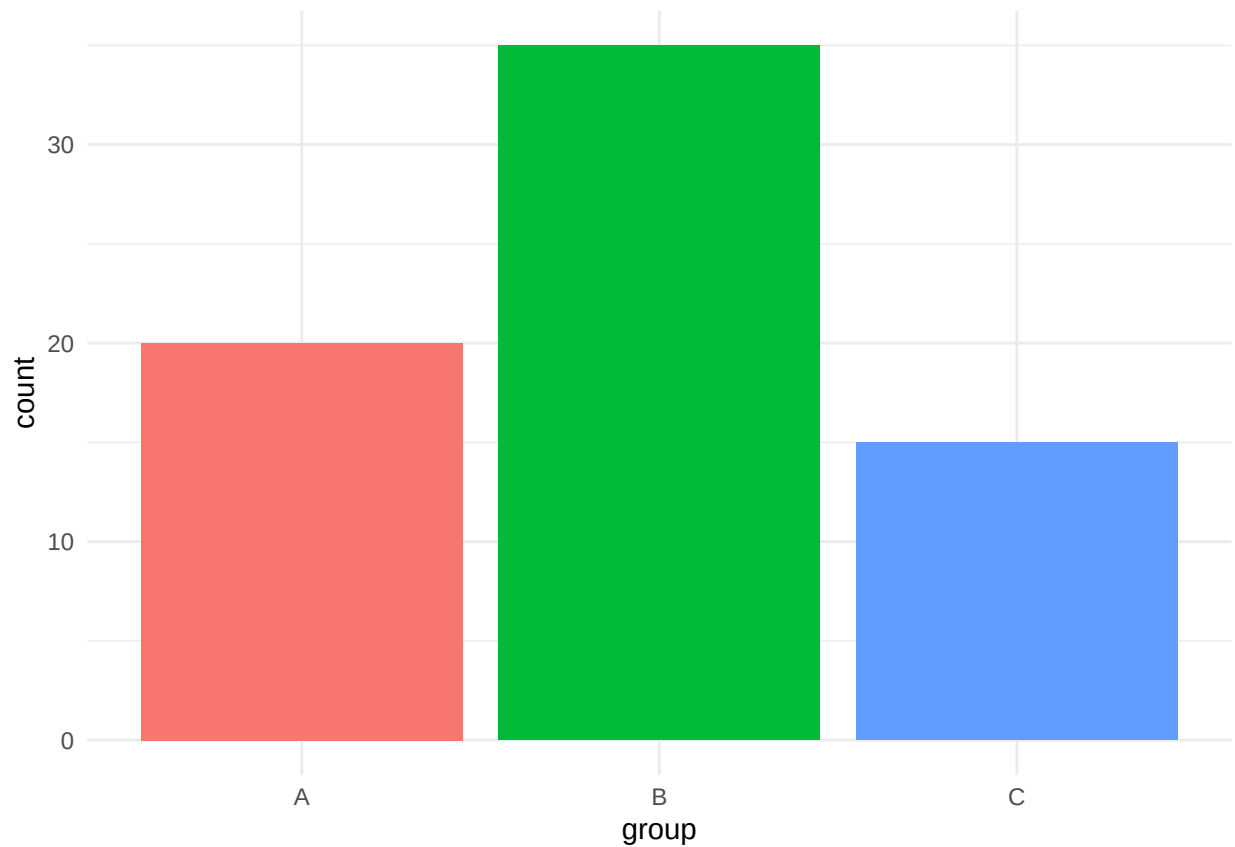


What to look for: linearity, outliers, clusters, heteroskedasticity (funnel shape).

24.4 22.4 Bar chart

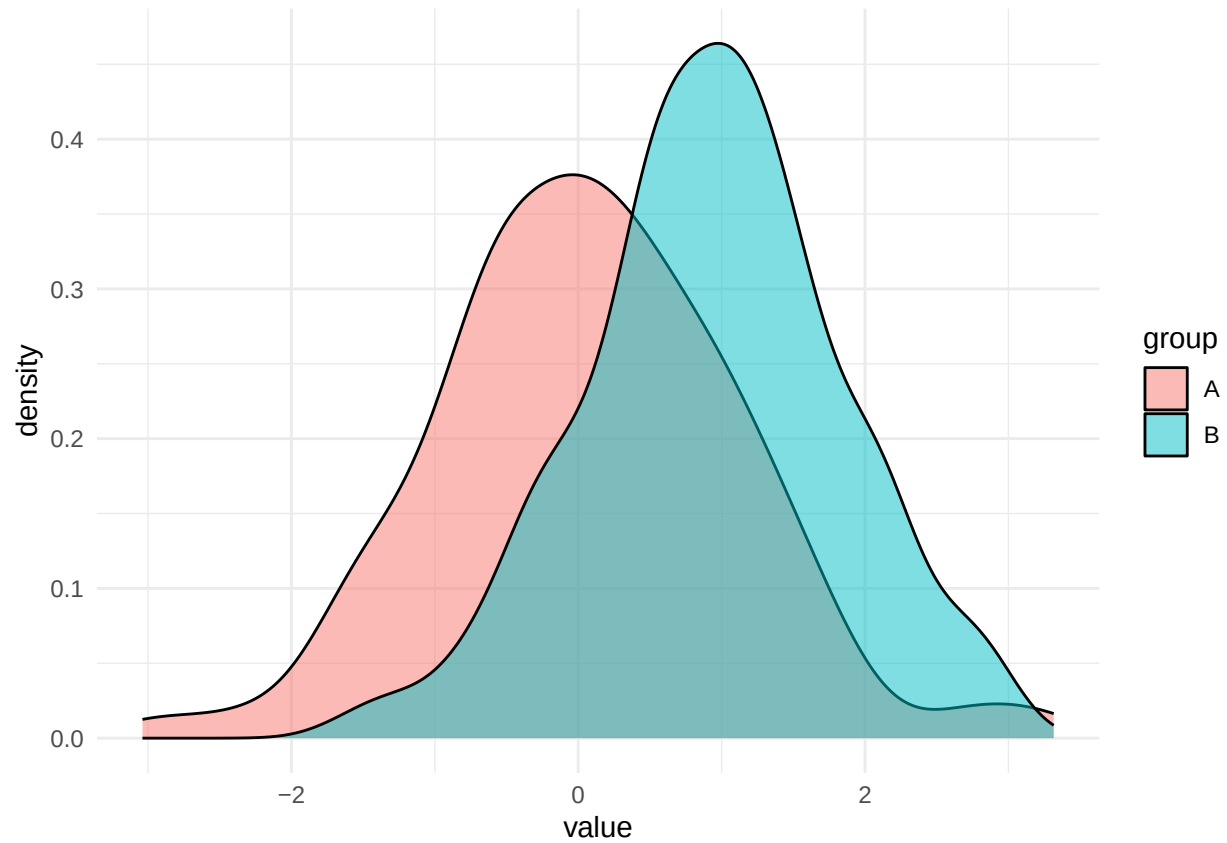
```
# --- Self-contained: bar chart ---
p22_df <- tibble(group = c("A", "B", "C"), count = c(20, 35, 15))

ggplot(p22_df, aes(group, count, fill = group)) +
  geom_col() +
  theme_minimal() +
  theme(legend.position = "none")
```



24.5 22.5 Density plot

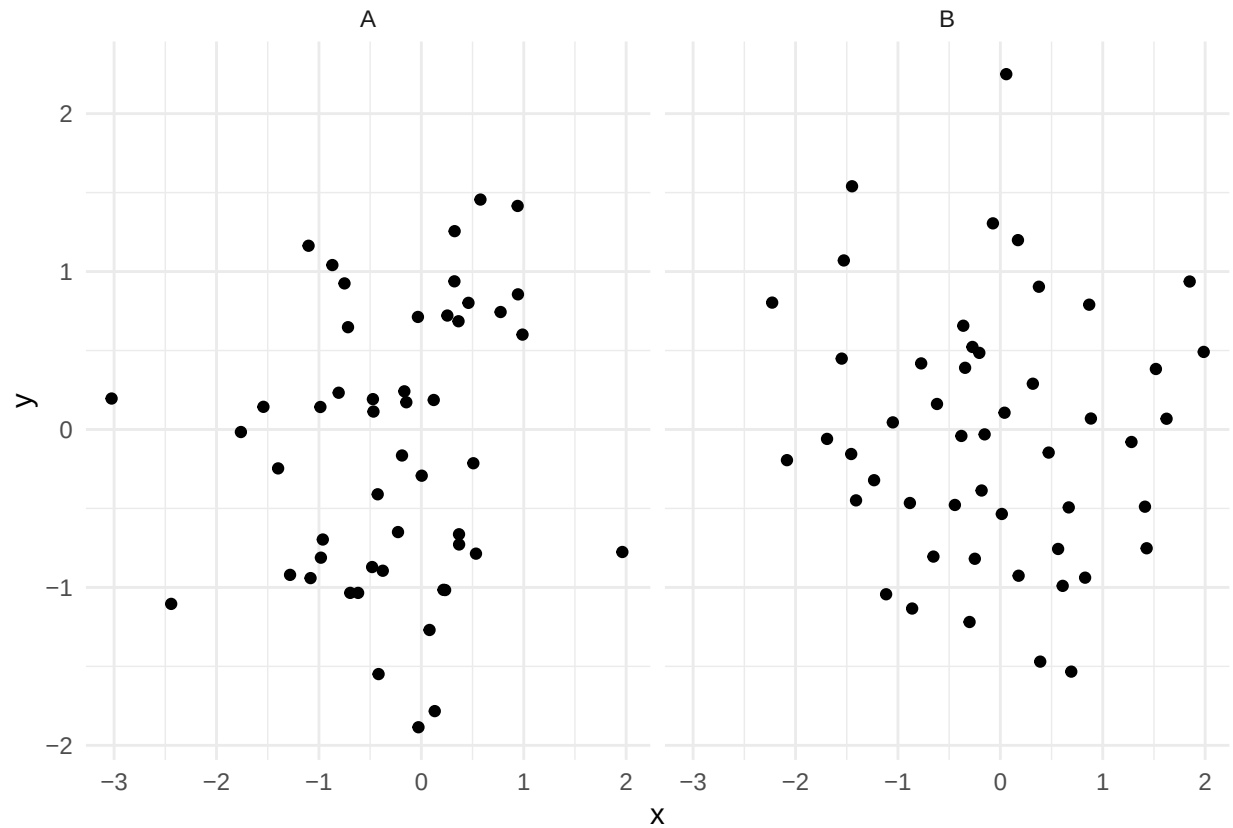
```
# --- Self-contained: density ---  
set.seed(2205)  
p22_df <- tibble(  
  group = rep(c("A", "B"), each = 200),  
  value = c(rnorm(200, 0), rnorm(200, 1))  
)  
  
ggplot(p22_df, aes(value, fill = group)) +  
  geom_density(alpha = 0.5) +  
  theme_minimal()
```



24.6 22.6 Facets

```
# --- Self-contained: faceted scatterplot ---
set.seed(2206)
p22_df <- tibble(
  x = rnorm(100),
  y = rnorm(100),
  group = rep(c("A", "B"), 50)
)

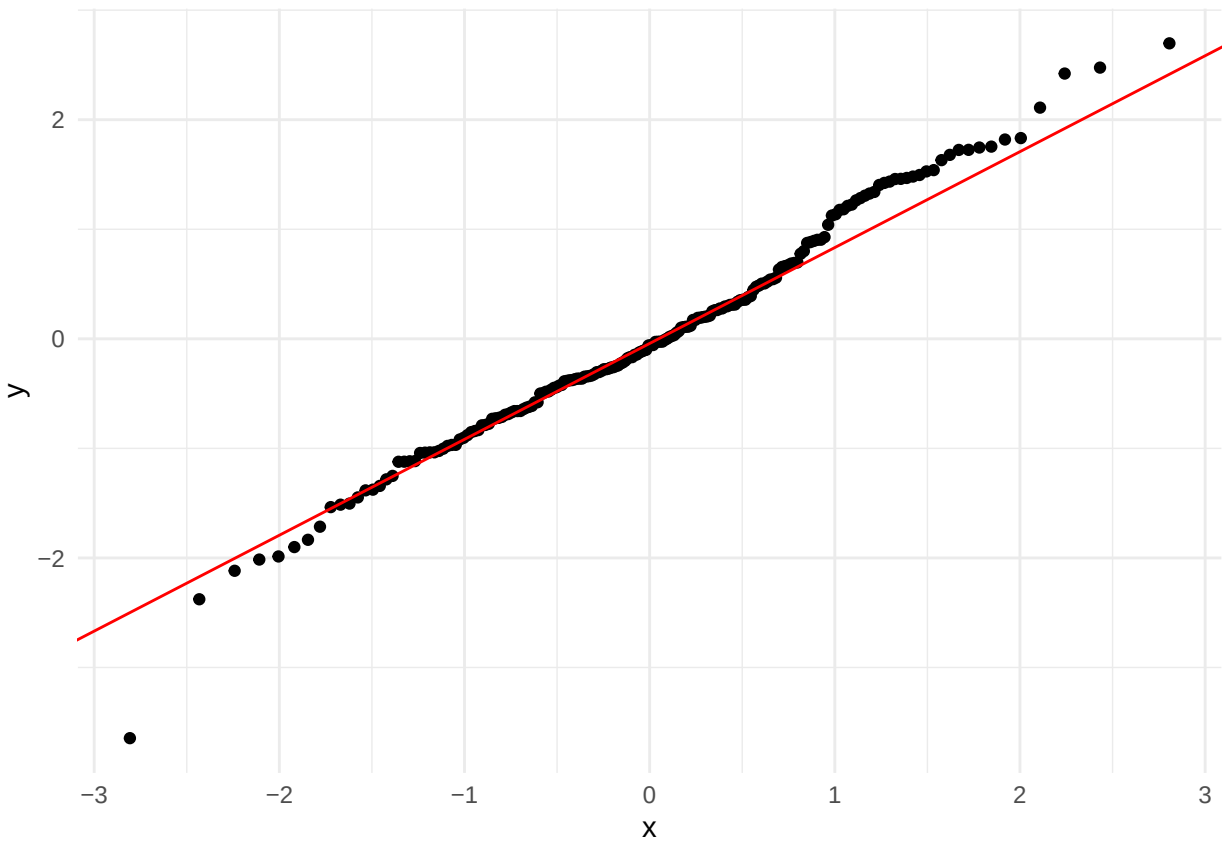
ggplot(p22_df, aes(x, y)) +
  geom_point() +
  facet_wrap(~ group) +
  theme_minimal()
```



24.7 22.7 QQ plot (normality check)

```
# --- Self-contained: QQ plot ---
set.seed(2207)
p22_df <- tibble(score = rnorm(200))

ggplot(p22_df, aes(sample = score)) +
  stat_qq() +
  stat_qq_line(color = "red") +
  theme_minimal()
```

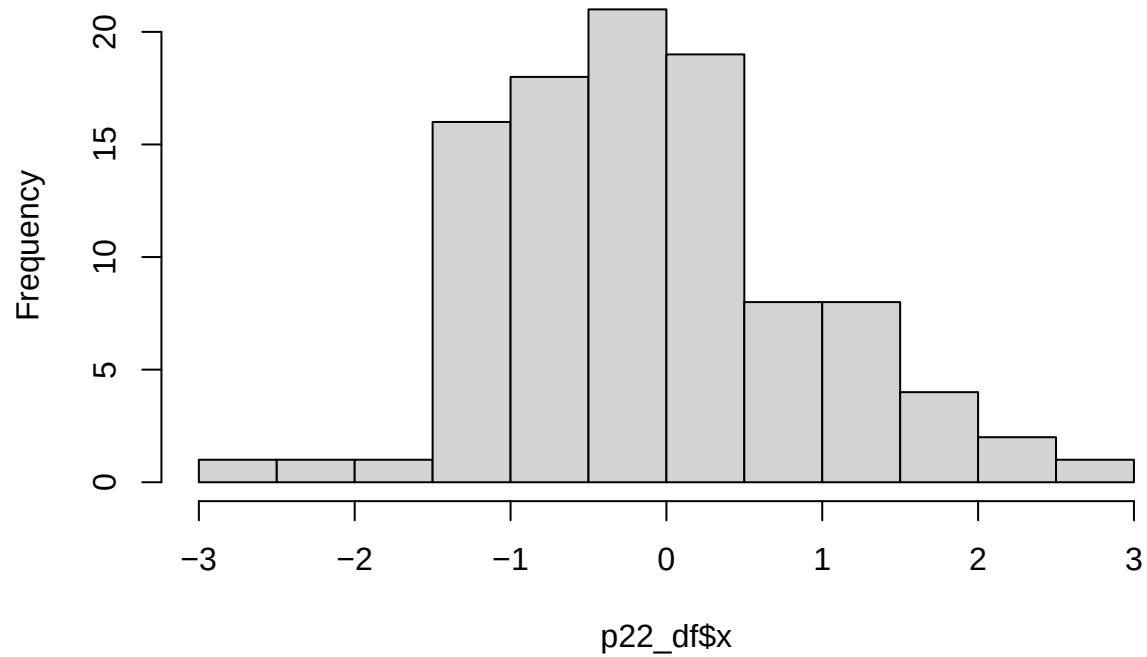


What to look for: points should follow the diagonal. Curvature = non-normal.

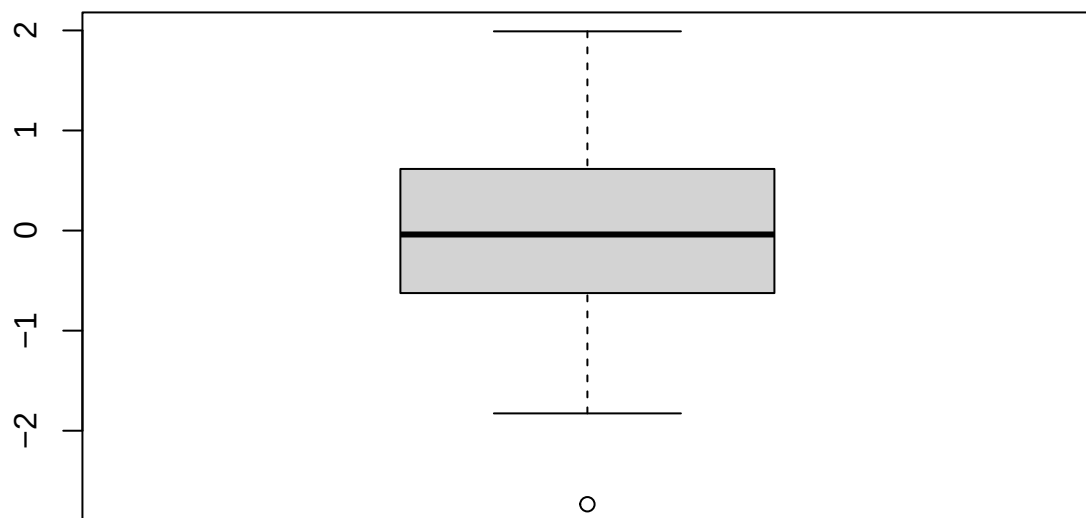
24.8 22.8 Base R alternatives

```
# --- Self-contained: base R plots ---  
set.seed(2208)  
p22_df <- tibble(x = rnorm(100), y = rnorm(100))  
  
hist(p22_df$x)
```

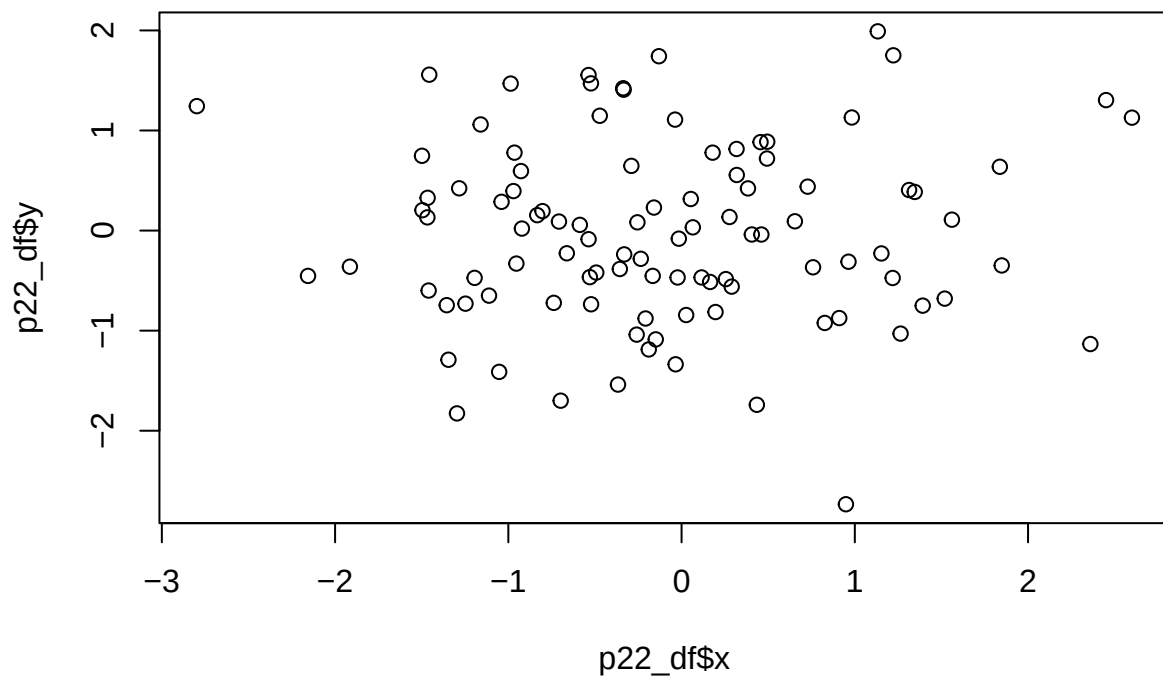
Histogram of p22_df\$x



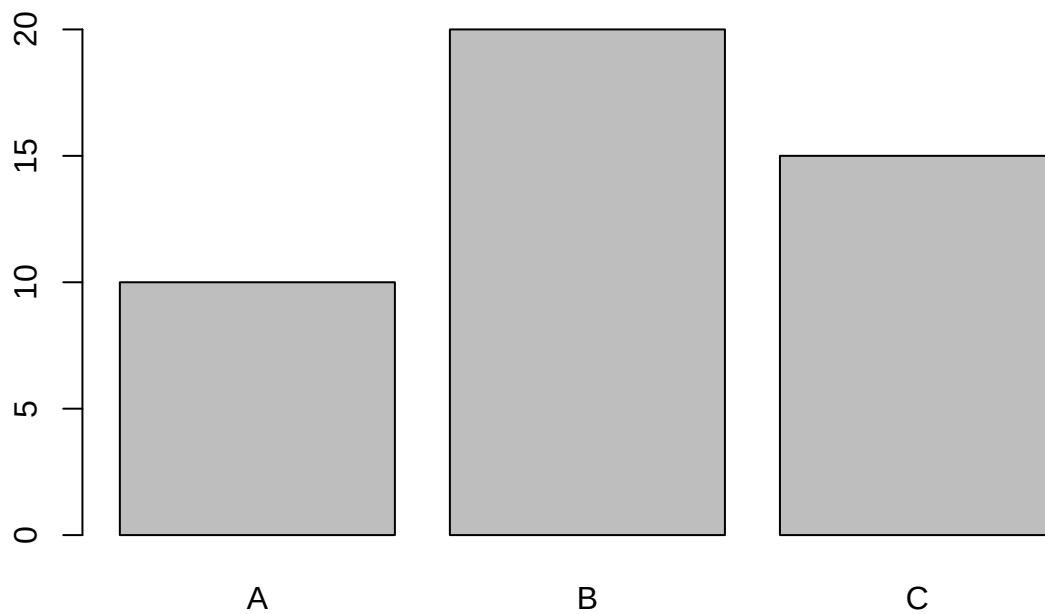
```
boxplot(p22_df$y)
```



```
plot(p22_df$x, p22_df$y)
```



```
barplot(c(A = 10, B = 20, C = 15))
```



24.9 22.9 Report template

“The boxplot shows that group C has the highest median (90.2) and group A the lowest (70.1). All groups have similar spread. There are no extreme outliers. The distribution of scores is approximately symmetric in each group.”

25 Part 23 — Export

25.1 23.1 CSV

```
write.csv(df, "out.csv", row.names = FALSE)
write_csv(df, "out.csv") # tidyverse, faster
```

25.2 23.2 Excel

```
library(writexl)
write_xlsx(df, "out.xlsx")
write_xlsx(list(Sheet1 = df1, Sheet2 = df2), "multi.xlsx")
```

25.3 23.3 RDS (R’s native format)

```
saveRDS(df, "df.rds")
df <- readRDS("df.rds")
```

RDS preserves all types (factors, dates, list-columns). Use it when your next R session will read it.

25.4 23.4 RData (multiple objects)

```
save(df, fit, file = "workspace.RData")
load("workspace.RData")
```

25.5 23.5 Text file

```
write.table(df, "out.txt", sep = "\t", row.names = FALSE, quote = FALSE)
```

25.6 23.6 Report tables

```
# --- Self-contained: kable table ---
p23_df <- tibble(group = c("A", "B"), mean = c(85.3, 84.0), sd = c(6.4, 11.5))
kable(p23_df, digits = 2, caption = "Mean and SD by group")
```

Table 43: Mean and SD by group

group	mean	sd
A	85.3	6.4
B	84.0	11.5

For reports: `kable()` for HTML/PDF, `kableExtra` for more styling.

26 Part 24 — Full Workflow Template

Copy-paste for any exam data-handling question.

```
# ---- 1. Load packages ----
library(tidyverse)
library(readxl)
library(knitr)

# ---- 2. Read ----
df <- read_excel("Q1.xls", sheet = 1)

# ---- 3. Inspect ----
dim(df)
str(df)
head(df)
colSums(is.na(df))

# ---- 4. Fix types ----
df$group <- factor(df$group, levels = c("control", "treat1", "treat2"))
df$y <- as.numeric(df$y)
```

```

df$date <- as.Date(df$date)

# ---- 5. Handle missing ----
df <- df %>% drop_na(y)

# ---- 6. Transform ----
df <- df %>%
  mutate(
    log_y = log(y),
    y_cat = cut(y, breaks = c(0, 50, 100, Inf), labels = c("low", "mid", "high"))
  )

# ---- 7. Reshape if needed ----
# df_long <- df %>% pivot_longer(...)

# ---- 8. Summarise ----
df %>%
  group_by(group) %>%
  summarise(n = n(), mean = mean(y), sd = sd(y), .groups = "drop") %>%
  kable(digits = 3)

# ---- 9. Plot ----
ggplot(df, aes(group, y, fill = group)) +
  geom_boxplot() +
  theme_minimal() +
  theme(legend.position = "none")

# ---- 10. Export ----
write_csv(df, "cleaned.csv")

```

27 Part 25 — Common Exam Scenarios

27.1 25.1 Excel with header rows to skip

```
df <- read_excel("Q1.xls", skip = 2)
```

27.2 25.2 Numeric-coded groups → factor with custom reference

```

# --- Self-contained: numeric codes to factor ---
p25_df <- tibble(id = 1:6, group = c(1, 2, 3, 1, 2, 3))
p25_df$group <- factor(p25_df$group, levels = c(3, 1, 2))
levels(p25_df$group)

```

```
## [1] "3" "1" "2"
```

27.3 25.3 Group-level summary table

```

# --- Self-contained: group summary ---
p25_df <- tibble(
  group = c("A", "A", "B", "B", "C", "C"),

```

```

  y      = c(10, 12, 20, 22, 30, 32)
)

p25_df %>%
  group_by(group) %>%
  summarise(n = n(), mean = mean(y), sd = sd(y), .groups = "drop") %>%
  kable(digits = 3)

```

group	n	mean	sd
A	2	11	1.414
B	2	21	1.414
C	2	31	1.414

27.4 25.4 Contingency table with percentages

```

# --- Self-contained: contingency percentages ---
p25_df <- tibble(
  gender = c("M", "M", "F", "F", "M", "F"),
  result = c("yes", "no", "yes", "yes", "no", "yes")
)

tab <- table(p25_df$gender, p25_df$result)
addmargins(tab)

```

```

##
##      no yes Sum
##  F    0  3   3
##  M    2  1   3
##  Sum  2  4   6

```

```

round(prop.table(tab, margin = 1) * 100, 1)

```

```

##
##      no  yes
##  F   0.0 100.0
##  M  66.7  33.3

```

27.5 25.5 Train / test split

```

# --- Self-contained: split ---
set.seed(2505)
p25_df <- tibble(id = 1:10, y = rnorm(10))
p25_idx <- sample(1:nrow(p25_df), size = floor(0.8 * nrow(p25_df)))
p25_train <- p25_df[p25_idx, ]
p25_test  <- p25_df[-p25_idx, ]
nrow(p25_train); nrow(p25_test)

```

```
## [1] 8
```

```
## [1] 2
```

27.6 25.6 Repeated measures wide → long

```
# --- Self-contained: repeated measures ---
p25_df <- tibble(
  id = 1:3,
  y0 = c(10, 12, 14),
  y1 = c(11, 13, 15),
  y2 = c(12, 14, 16)
)

p25_long <- p25_df %>%
  pivot_longer(cols = starts_with("y"),
               names_to = "time",
               values_to = "score")
kable(p25_long)
```

id	time	score
1	y0	10
1	y1	11
1	y2	12
2	y0	12
2	y1	13
2	y2	14
3	y0	14
3	y1	15
3	y2	16

27.7 25.7 Treat a specific value as missing

```
# --- Self-contained: recode to NA ---
p25_df <- tibble(id = 1:5, y = c(10, -99, 14, 16, -99))
p25_df$y[p25_df$y == -99] <- NA
kable(p25_df)
```

id	y
1	10
2	NA
3	14
4	16
5	NA

27.8 25.8 Duplicate rows

```
# --- Self-contained: duplicates ---
p25_df <- tibble(id = c(1, 1, 2, 3, 3), y = c(10, 10, 20, 30, 30))

p25_df %>% distinct() # remove exact duplicates

## # A tibble: 3 x 2
##   id     y
##   <dbl> <dbl>
```

```
## 1      1      10
## 2      2      20
## 3      3      30

p25_df %>% distinct(id, .keep_all = TRUE) # first row per id
```

```
## # A tibble: 3 x 2
##       id      y
##   <dbl> <dbl>
## 1     1    10
## 2     2    20
## 3     3    30
```

```
deduplicated(p25_df) # TRUE/FALSE per row
```

```
## [1] FALSE TRUE FALSE FALSE TRUE
```

27.9 25.9 Wide-to-long with multiple value columns

```
# --- Self-contained: pivot_longer with multiple values ---
p25_df <- tibble(
  id = 1:3,
  y0_val = c(10, 12, 14), y1_val = c(11, 13, 15),
  y0_sd = c(1, 1.1, 1.2), y1_sd = c(1.1, 1.2, 1.3)
)

p25_df %>%
  pivot_longer(
    cols = starts_with("y"),
    names_to = c("time", ".value"),
    names_sep = "_"
  ) %>%
  kable()
```

id	time	val	sd
1	y0	10	1.0
1	y1	11	1.1
2	y0	12	1.1
2	y1	13	1.2
3	y0	14	1.2
3	y1	15	1.3

27.10 25.10 Left-join with missing keys

```
# --- Self-contained: left join with missing ---
p25_a <- tibble(id = 1:4, x = c("a", "b", "c", "d"))
p25_b <- tibble(id = c(2, 4), y = c("P", "Q"))

left_join(p25_a, p25_b, by = "id")
```

```
## # A tibble: 4 x 3
##       id x      y
##   <dbl> <chr> <chr>
## 1     1 a    <NA>
```

```
## 2      2 b      P
## 3      3 c      <NA>
## 4      4 d      Q
```

Rows 1 and 3 get NA for y.

27.11 25.11 Rounded count table

```
# --- Self-contained: rounded counts ---
p25_df <- tibble(group = c("A", "B", "C"), n = c(20, 35, 15))
p25_df$pct <- round(100 * p25_df$n / sum(p25_df$n), 1)
kable(p25_df)
```

group	n	pct
A	20	28.6
B	35	50.0
C	15	21.4

28 Part 26 — Quick Reference Table

Task	Command
Create df	<code>data.frame(a, b, c) / tibble(...) / tribble(...)</code>
Read Excel	<code>read_excel("f.xls", sheet = 1)</code>
Read CSV	<code>read_csv("f.csv")</code>
Read SPSS	<code>read_sav("f.sav")</code>
Inspect	<code>dim, str, glimpse, head, summary</code>
NA count	<code>colSums(is.na(df))</code>
Convert type	<code>as.numeric, as.factor, as.character</code>
Set reference	<code>factor(x, levels = c("ref", ...))</code>
Change reference	<code>relevel(x, ref = "ref")</code>
Check dummies	<code>contrasts(df\$group)</code>
Manual dummy	<code>ifelse(x == "A", 1, 0)</code>
Select columns	<code>select(a, b), select(starts_with("q"))</code>
Drop column	<code>select(-id)</code>
Filter rows	<code>filter(x > 5), filter(g %in% c("A", "B"))</code>
New column	<code>mutate(z = x + y)</code>
Conditional column	<code>mutate(z = if_else(x > 5, "hi", "lo"))</code>
Multi-condition	<code>case_when(...)</code>
Categorize	<code>cut(x, breaks = ..., labels = ...)</code>
Rename	<code>rename(new = old)</code>
Reorder	<code>relocate(score, .after = id)</code>
Sort	<code>arrange(desc(x))</code>
Group summary	<code>group_by(g) %>% summarise(m = mean(x))</code>
Count	<code>count(g) / table(x)</code>
Frequency %	<code>prop.table(table(x))</code>
Wide to long	<code>pivot_longer(...)</code>
Long to wide	<code>pivot_wider(...)</code>
Stack rows	<code>bind_rows(a, b)</code>
Join	<code>left_join(a, b, by = "id")</code>

Task	Command
Split data	<code>sample(), set.seed()</code>
Remove NA	<code>drop_na(df) / filter(!is.na(x))</code>
Replace NA	<code>x[is.na(x)] <- mean(x, na.rm = TRUE)</code>
Export	<code>write_csv(df, "out.csv")</code>
Report table	<code>kable(df, digits = 2)</code>
String ops	<code>str_detect, str_replace, str_trim</code>
Dates	<code>as.Date, format, lubridate::ymd</code>

29 Part 27 — Theory Q&A (What to Say)

Q: What is a data frame? A: A table with rows (observations) and columns (variables). Each column is a vector of the same length, but columns can be of different types. It's R's main structure for data.

Q: What is tidy data? A: A data frame where each variable is a column, each observation is a row, and each value is a cell. Most tidyverse functions expect this format.

Q: What's the difference between `data.frame` and `tibble`? A: Tibbles print nicer, don't auto-convert strings to factors, don't use row names, and support list-columns. Use tibbles.

Q: What's the difference between `[`, `[[`, and `$`? A: `[` returns a subset (still a tibble). `[[` and `$` return the contents (a vector).

Q: Why convert a numeric-coded group to a factor? A: Regression and ANOVA treat factors as categorical and create dummy variables. If you leave numeric codes, R will treat them as a continuous predictor, giving completely wrong results.

Q: Why does the reference level matter? A: In regression, the reference is the baseline. Coefficients for other levels are interpreted as differences from the reference. If you set the wrong reference, interpretation changes.

Q: What's the difference between removing and imputing missing values? A: Removing loses data and can bias results if missingness is systematic. Imputing keeps sample size but introduces assumptions. Use removal if <5% missing and MCAR; impute otherwise.

Q: What's the difference between NA, NaN, NULL, Inf? A: NA is missing, propagates through operations. NaN is "not a number" (0/0), is also NA. NULL is empty, has length 0. Inf is infinity, treated as a real value.

Q: What's the difference between `filter` and `subset`? A: `filter` is from dplyr, works in pipes, uses `,` for AND. `subset` is base R, uses `&` for AND, can select columns.

Q: What's the difference between `ifelse` and `if_else`? A: `ifelse` is base R, evaluates both branches, and drops NA to NA. `if_else` is dplyr, type-checks both branches, and lets you specify the missing value.

Q: Why use `group_by` before `summarise`? A: `group_by` splits the data by a variable. `summarise` then computes one row per group. Without `group_by`, `summarise` computes over all rows.

Q: What's the difference between wide and long format? A: Wide: one row per subject, columns for each measurement. Long: one row per measurement per subject. Long is needed for repeated-measures models and ggplot with multiple series.

Q: What is the difference between the six join types? A: `inner_join` keeps matching rows only. `left_join` keeps all rows from the left. `right_join` keeps all from the right. `full_join` keeps all from both. `anti_join` keeps rows in left not in right. `semi_join` keeps rows in left that match right (left columns only).

Q: How do you decide what to do with outliers? A: First check if it's a data-entry error. If yes, remove. If no, compute analysis both with and without it, and report both. Never remove silently.

Q: What's the difference between `table()` and `xtabs()`? A: `table()` gives counts of one or more factors. `xtabs()` is a formula-based interface, useful when you have counts already summed.

Q: How do you split data for cross-validation? A: Use `set.seed()` for reproducibility, then `sample(1:n, size = ...)` to draw row indices. Use the indices for training, the rest for testing.

30 Part 28 — Common Mistakes

1. **Not checking `str(df)`** — a numeric column read as character breaks everything downstream.
 2. **Forgetting `na.rm = TRUE`** — `mean(x)` returns NA if any value is missing.
 3. **Using `=` instead of `==` inside `filter()`** — R assigns instead of comparing.
 4. **Treating a factor as numeric** — `as.numeric(factor)` returns level codes, not values. Use `as.numeric(as.character(x))`.
 5. **Not setting the reference level** — regression output becomes uninterpretable.
 6. **Not setting `set.seed()` before `sample()`** — different split each knit.
 7. **Overwriting variables** — `df <- df %>% filter(...)` is fine; accidentally reassigning an intermediate variable causes silent bugs.
 8. **Silent 0-row results** — always check `nrow()` after filtering.
 9. **Forgetting `library(tidyverse)`** — `%>%`, `mutate`, `filter` won't work.
 10. **Length mismatch in assignment** — `df$col <- c(1, 2, 3)` on a 10-row data frame will error or recycle incorrectly.
 11. **Reusing a variable name from an earlier chunk** — stale data, wrong results. Use section-prefixed names.
 12. **Pivoting without checking for duplicates** — `pivot_wider` fails or creates list-columns if the `id` + key combination isn't unique.
 13. **Comparing factors by `==` with numbers** — factors compare by level, not value. Convert first.
 14. **`read_csv` importing dates as character** — check with `str()`, convert with `as.Date`.
 15. **Forgetting `row.names = FALSE` in `write_csv`** — creates an extra unnamed column when read back.
 16. **Assuming `bind_rows` matches by name** — it does match by name, but `cbind` matches by position.
 17. **Using `mean()` on a factor** — returns NA with a warning.
 18. **Not verifying row counts before and after a join** — a many-to-many join can duplicate rows.
 19. **Silently removing NAs with `na.omit`** — always report how many rows were removed.
 20. **Naming a variable the same as a function** — `mean <- c(1, 2, 3)` breaks `mean(x)`.
-

31 Part 29 — The Single Most Important Idea

Every data-handling question is one of these:

1. **Read** the data (Excel, CSV, SPSS, text).
2. **Inspect** (`dim`, `str`, `head`, NA counts, unique values).
3. **Fix types** (numeric, factor with reference, Date).
4. **Clean** (drop NA, filter, recode).
5. **Transform** (`mutate`, `cut`, `pivot`).
6. **Summarise** (`group_by`, `summarise`, `count`).
7. **Verify** (row counts, spot-checks).
8. **Report** in words: what you did, why, and what the output means.

If you can do these eight steps, you can handle any data-handling question in the exam.

End of cheat sheet. Good luck in the competition. ““

31.1 What was added and why

Addition	Why it matters
Part 0 — 30-second strategy	Immediate action plan for any question.
Part 1 — Foundations	Tidy data principles, tibble vs data.frame, \$ vs [] vs [.
Part 2 — Creating data (6 methods)	data.frame, tibble, tribble, matrix, matrix of counts, lists, sequences.
Part 3 — Import (7 sources)	Excel, SPSS, SAS, Stata, text, clipboard, URLs — with common pitfalls.
Part 4 — Inspection (5 commands)	The “always run these” checklist.
Part 5 — Data types (7 types)	Numeric, character, logical, factor, ordered factor, Date, POSIXct — with detection and conversion.
Part 6 — Type conversion	Includes the factor→numeric trap.
Part 7 — Factors in depth	Reference levels, relevel, contrasts, ordered, recode, fct_lump.
Part 8 — Missing values (full)	Detect, remove, replace, interpolate, LOCF, na.rm, NA vs NaN vs NULL vs Inf.
Part 9 — Column ops	select, rename, relocate, pull, rename_with.
Part 10 — Row ops	filter, slice, distinct, arrange, with row-count verification.
Part 11 — Mutating	mutate, if_else, case_when, cut, across, transmute.
Part 12 — Grouped	group_by + summarise, grouped mutate, grouped filter, count.
Part 13 — Window functions	lag, lead, cumsum, rankings, n(), first/last.
Part 14 — Reshaping	pivot_longer, pivot_wider, separate, unite, complete.
Part 15 — Combining	bind_rows, bind_cols, all six joins, joining by different names.
Part 16 — Frequency tables	One-way, two-way, chi-square, CrossTable.
Part 17 — Descriptive by group	Summary tables, SE, CI, formatted for reports.
Part 18 — Sampling	Simple, train/test, stratified, K-fold.
Part 19 — Outliers	IQR, Z-score, winsorize, remove.
Part 20 — Strings	Base R + stringr, common cleaning tasks.
Part 21 — Dates	as.Date, formats, arithmetic, lubridate.
Part 22 — Plots	Histogram, boxplot, scatter, bar, density, facets, QQ.
Part 23 — Export	CSV, Excel, RDS, RData, text, kable.
Part 24 — Full workflow template	Copy-paste for any exam question.
Part 25 — Common exam scenarios	11 patterns drawn from typical exams.
Part 26 — Quick reference table	One-liner answers.
Part 27 — Theory Q&A	Ready-made answers for conceptual questions.
Part 28 — Common mistakes	20 traps to avoid.
Part 29 — Single most important idea	Eight-step pipeline.

Design consistency: Every chunk is self-contained with section-prefixed names (p3_df, p7_fac, p14_long), so you can run any chunk in isolation. Copy-paste any chunk into a fresh R session and it works. Run the whole file top-to-bottom — no errors.